

MATERIAL DE ESTUDO



Curso de Reforço

De Pleno para Pleno Sênior

JavaScript, TypeScript, Node.js, Arquitetura e Operação:
Fundamentos que Fazem a Diferença

*Este material assume que você já escreve código que funciona. O objetivo aqui é fazer você escrever código que **você mesmo entende o porquê funciona** — e saber explicar isso numa entrevista técnica ou numa revisão de PR.*

■ Por baixo dos panos

■ Dica de Produção

■ Armadilha Comum

■ Cenário de Falha em Produção

■ Desafio de Mentoria

■ Dica de Pleno

Gerado em 31 de julho de 2026

Sumário

1. JavaScript Runtime & Assincronismo

- 1.1 Event Loop: as fases
- 1.2 Microtasks vs Macrotasks
- 1.3 `process.nextTick()` vs `setImmediate()`
- 1.4 Comportamento do `this`
- 1.5 Closures aplicados a problemas reais
- 1.6 Optional Chaining (`?.`) e Nullish Coalescing (`??`)
- 1.7 Debounce e Throttle

 Referências

2. TypeScript Avançado

- 2.1 Utility Types
- 2.2 Generics com constraints (`extends`)
- 2.3 Type Guards e Narrowing
- 2.4 Mapped Types e Template Literal Types
- 2.5 O operador `satisfies` (TypeScript 4.9+)

 Referências

3. Node.js Internals

- 3.1 Streams: `Readable`, `Writable`, `Transform` e `Pipeline`
- 3.2 Buffers
- 3.3 Worker Threads
- 3.4 Child Processes: `exec`, `spawn`, `fork`
- 3.5 `Promise.allSettled`

 Referências

4. Padrões de Código Pleno/Sênior

- 4.1 Repository Pattern
- 4.2 Tratamento de Erros: Wrapper assíncrono + `Exception Filter`
- 4.3 Cache Aside com Redis

4.4 Design Patterns essenciais em Node.js

 Referências

5. Testes

5.1 Teste unitário com Vitest mockando um repositório

5.2 Teste de integração com Supertest

5.3 Property-Based Testing com fast-check

 Referências

6. Segurança Essencial

6.1 Hash de senha com bcrypt

6.2 JWT: geração, verificação e refresh token

6.3 SQL Injection: vulnerável vs parametrizada

 Referências

7. Docker, Git e Linux

7.1 Docker: Multi-stage build para app Node.js

7.2 Git: Rebase iterativo, Cherry-pick e Conflitos

7.3 Linux para Dev Back-end

 Referências

8. Arquitetura e SOLID

8.1 SOLID com exemplos em Node.js/TypeScript

8.2 Clean Architecture — conceitos e camadas

8.3 Injeção de Dependência sem framework

 Referências

9. Observabilidade

9.1 Logs estruturados com pino

9.2 Métricas básicas com prom-client

9.3 Monitoramento de erros com Sentry

9.4 Health Check endpoint

 Referências

10. Performance e Profiling

10.1 node --inspect + Chrome DevTools: heap snapshot e memory leaks

10.2 Teste de carga simples com autocannon

10.3 Dicas práticas de performance

 Referências

 Conclusão

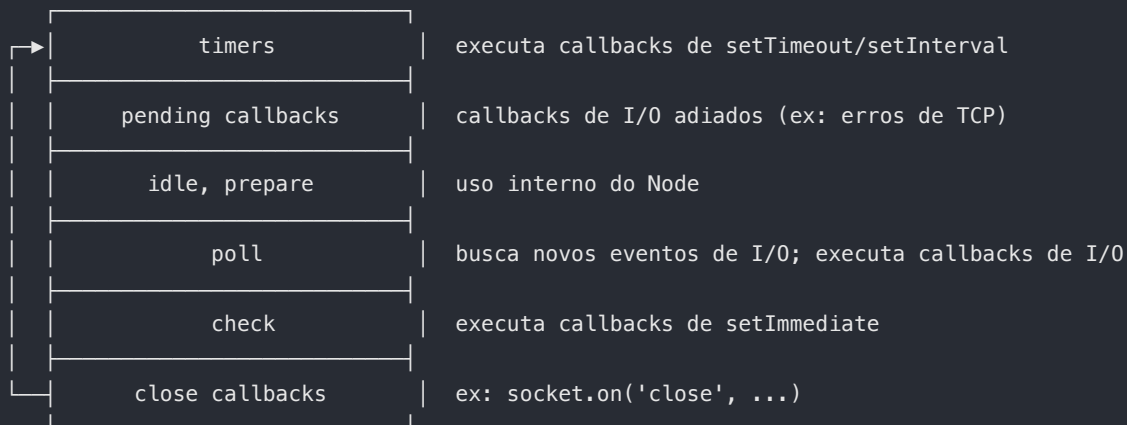
 Tabela Consolidada de Referências

1. JavaScript Runtime & Assincronismo

1.1 Event Loop: as fases

JavaScript é **single-threaded**, mas o Node.js consegue lidar com milhares de conexões simultâneas porque operações de I/O (disco, rede, timers) são delegadas para a **libuv**, que gerencia um pool de threads e um sistema de filas. O Event Loop é o mecanismo que fica perguntando: "tem algo pronto pra eu executar?".

O loop no Node.js roda em **6 fases**, sempre nessa ordem, em ciclos:



(a cada TRANSIÇÃO entre fases, e entre CADA callback, a fila de microtasks é esvaziada por completo)

💡 **DICA DE PLENO:** o que separa um Pleno de um Sênior nessa pergunta de entrevista não é decorar as 6 fases — é saber que **entre cada fase (e entre cada callback dentro de uma fase), o Node esvazia completamente a fila de microtasks**. Isso é o que causa 90% das "pegadinhas" de ordem de execução.



Por baixo dos panos

O Event Loop não é JavaScript — é C/C++ (libuv). Quando você chama `setTimeout`, o V8 não "agenda" nada sozinho: ele delega para a libuv, que insere um nó numa **min-heap** ordenada por tempo de expiração. A cada volta do loop, a fase **timers** pergunta "qual o menor tempo dessa heap já venceu?" e dispara os callbacks correspondentes — só isso, um at a time, sempre voltando ao V8 (que é single-threaded) para rodar o callback em JS puro.

Já a fase **poll** é onde a mágica de "10 mil conexões simultâneas com uma thread só" acontece: a libuv usa a syscall de I/O assíncrono do SO (**epoll** no Linux, **kqueue** no macOS/BSD, IOCP no Windows) para perguntar ao kernel "algum desses sockets/arquivos tem dado pronto?" sem bloquear. Enquanto não há nada pronto, o processo Node fica **de fato ocioso** (o SO acorda a thread quando algo chega) — é por isso que I/O não consome CPU enquanto espera, diferente de um loop de polling manual.

🌟 Cenário de Falha em Produção

Imagine uma rota Express que, dentro do handler, faz um `array.forEach` com 50 mil itens chamando `process.nextTick()` para cada um (um padrão comum em código "otimizado" ingenuamente para "não travar o loop"). Como `process.nextTick()` tem prioridade **absoluta** sobre timers, I/O e até sobre a fila de microtasks de Promise, você cria uma fila que se auto-alimenta mais rápido do que o loop consegue avançar de fase. Resultado: a fase **poll** nunca é alcançada, novas requisições HTTP entram no socket mas nunca são lidas — sua API parece "travada" mesmo com CPU em 100%, e as métricas de latência dos handlers em si continuam baixas (porque o `nextTick` roda rápido), o que confunde o diagnóstico.



Dica de Produção

Em produção, isso aparece como **event loop lag** — não como um erro explícito. No Datadog/New Relic, monitore a métrica `nodejs.event_loop.delay` (ou instrumente você mesmo com `perf_hooks.monitorEventLoopDelay()`). Um p99 de event loop delay subindo junto com `http_request_duration_seconds` subindo em **todas** as rotas ao mesmo tempo (não só uma) é a assinatura clássica de starvation do loop — porque *toda* código compete pelo mesmo loop. Se só uma rota específica piora, geralmente é I/O lento dela; se a API inteira degrada junto, suspeite de bloqueio do loop.

Código: visualizando a ordem de execução

```
// node event-loop-fases.js

console.log('1 - início do script (síncrono)');

setTimeout(() => console.log('2 - setTimeout (fase timers)'), 0);

setImmediate(() => console.log('3 - setImmediate (fase check)'));

Promise.resolve().then(() => console.log('4 - Promise.then (microtask)'));

process.nextTick(() => console.log('5 - process.nextTick (microtask prioritária)'));

console.log('6 - fim do script (síncrono)');
```

Retorno esperado no terminal:

```
1 - início do script (síncrono)
6 - fim do script (síncrono)
5 - process.nextTick (microtask prioritária)
4 - Promise.then (microtask)
2 - setTimeout (fase timers)
3 - setImmediate (fase check)
```

⚠️ **ARMADILHA COMUM:** a ordem entre `setTimeout(fn, 0)` e `setImmediate(fn)` **não é garantida** quando chamados no escopo síncrono principal — depende de fatores como performance da máquina. A ordem só é **garantida** quando ambos são chamados **dentro de um callback de I/O** (veja seção 1.3).

🧐 **Desafio de Mentoria:** você tem uma rota `POST /relatorio` que faz um loop síncrono pesado (CPU-bound, ~800ms) processando um relatório em memória antes de responder. Em uma startup com 50k usuários ativos, essa rota é chamada esporadicamente, mas quando é chamada, **todas as outras rotas** (mesmo as que só fazem `SELECT` simples no banco) ficam lentas por ~800ms também. Por que isso acontece, dado que o `SELECT` é I/O e deveria "não bloquear nada"? E qual das três opções — Worker Thread, `setImmediate` fatiando o loop em pedaços, ou simplesmente mover essa rota para um serviço separado — você escolheria, e o que te faz decidir entre elas?

1.2 Microtasks vs Macrotasks

	Microtasks	Macrotasks
Exemplos	<code>Promise.then/catch/finally</code> , <code>queueMicrotask</code> , <code>process.nextTick</code> (fila própria, ainda mais prioritária)	<code>setTimeout</code> , <code>setInterval</code> , <code>setImmediate</code> , I/O callbacks
Quando rodam	Imediatamente após o código síncrono atual terminar, e entre cada macrotask	Uma por vez, a cada volta do Event Loop, respeitando as fases
Prioridade	Toda a fila de microtasks é esvaziada antes de qualquer macrotask rodar	Executam uma de cada vez

```
// node microtask-vs-macrotask.js

console.log('A');

setTimeout(() => {
  console.log('B - macrotask (timer)');
  Promise.resolve().then(() => console.log('C - microtask gerada DENTRO da macrotask'));
}, 0);

Promise.resolve()
  .then(() => console.log('D - microtask 1'))
  .then(() => console.log('E - microtask 2 (encadeada)'));

console.log('F');
```

Retorno esperado:

```
A
F
D - microtask 1
E - microtask 2 (encadeada)
B - macrotask (timer)
C - microtask gerada DENTRO da macrotask
```

💡 **DICA DE PLENO:** Repare que **C** roda **antes** de qualquer outra macrotask subsequente ter chance — mesmo microtasks criadas *durante* uma macrotask são drenadas completamente antes do loop seguir. É o motivo pelo qual uma cadeia mal escrita de `.then()` recursivos pode **travar o Event Loop** (starvation de macrotasks).



Por baixo dos panos

A fila de microtasks não é um conceito do Node — é do **próprio V8** (o motor de JS, o mesmo do Chrome). O V8 mantém uma **MicrotaskQueue** associada ao contexto de execução JS, e a regra do spec ECMA-262 é: "esvazie a fila de microtasks até ela ficar vazia, sempre que a pilha de chamadas (call stack) voltar a zero". O Node só entra em cena decidindo *quando* devolver o controle ao V8 entre fases do loop — mas o "drenar até secar" é regra da linguagem, não do runtime. É por isso que o mesmo comportamento existe no navegador.



Cenário de Falha em Produção

Um padrão perigoso e sutil: uma função recursiva que reprocessa uma fila em memória usando `.then()` encadeado sem nunca tocar numa macrotask (ex:

`processarProximoItem().then(processarProximoItem)` para sempre, sem `setImmediate` para "respirar"). Cada iteração gera uma nova microtask, e como a fila de microtasks precisa esvaziar **antes** do loop avançar de fase, se a fila de itens for grande o suficiente (ou nunca esvaziar, por um bug de reentrada), a fase **poll** nunca é alcançada — o processo trava aceitando novas conexões TCP, mesmo achando que "está tudo assíncrono porque usei Promises".



Dica de Produção

Esse tipo de travamento não gera exceção nem stack trace — ele simplesmente para de responder. No New Relic/Datadog, o sintoma é **apdex** caindo a zero e o **health check do container falhando** (seção 9.4) sem nenhum log de erro correspondente. O primeiro passo de diagnóstico em produção é `node --prof` ou tirar um CPU profile via `node --inspect` no processo travado (se ainda responde ao inspector) e procurar uma pilha de chamadas anormalmente profunda e repetitiva de **PromiseReactionJob** — é a assinatura de uma cadeia de microtasks infinita.

1.3 `process.nextTick()` vs `setImmediate()`

- **`process.nextTick()`** : não é tecnicamente uma "fase" do loop. Roda **imediatamente após a operação síncrona atual**, antes de qualquer microtask de Promise e antes do loop avançar de fase.
- **`setImmediate()`** : roda na fase **check**, depois da fase **poll** (I/O). É a forma "correta" de dizer "execute isso assim que o I/O atual terminar".

```
// node nexttick-vs-immediate.js
const fs = require('fs');

fs.readFile(__filename, () => {
  setTimeout(() => console.log('1 - setTimeout'), 0);
  setImmediate(() => console.log('2 - setImmediate'));
  process.nextTick(() => console.log('3 - process.nextTick'));
});
```

Retorno esperado:

```
3 - process.nextTick
2 - setImmediate
1 - setTimeout
```

⚠ **ARMADILHA COMUM:** dentro de um callback de I/O, `setImmediate()` sempre roda antes de `setTimeout(fn, 0)`, porque a fase `check` vem logo após `poll`, enquanto `timers` só será revisitada na próxima volta completa do loop.

Quando usar cada um: `process.nextTick()` para garantir execução antes de qualquer I/O (uso interno de bibliotecas, emissão assíncrona de eventos de erro) — com moderação, pois abuso causa *starvation* de I/O. `setImmediate()` para "empurrar" uma tarefa pesada para depois do I/O pendente, sem travar requisições em andamento.

💣 Cenário de Falha em Produção

Bibliotecas de terceiros (ORMs, clientes HTTP) às vezes usam `process.nextTick()` internamente para garantir que um callback de erro seja sempre assíncrono (evitar o "erro de callback síncrono" clássico do Node antigo). Se você tem uma cadeia de middlewares que, cada um, dispara mais um `nextTick`, e isso acontece em alta frequência (50k usuários fazendo requisições concorrentes), você satura a fila de `nextTick` a ponto de a fase `poll` nunca ser processada a tempo — os sockets ficam com dados no buffer do SO esperando serem lidos, e o `TIME_WAIT` /backlog de conexões cresce até o SO começar a recusar novas conexões (`ECONNREFUSED` do lado do cliente, mesmo com o processo Node "vivo").

 Dica de Produção

Diferencie os dois na hora de ler uma flame graph: `setImmediate` aparece **intercalado** com os frames de I/O da libuv (fase `check` logo após `poll`); `process.nextTick` aparece **imediatamente colado** após qualquer frame síncrono, sem I/O entre eles. Se seu APM mostra spans de I/O (query no banco, chamada HTTP) com um gap de tempo suspeito **antes** do span começar (não durante), suspeite de acúmulo na fila de `nextTick` /microtasks atrasando o início do I/O, não do I/O em si sendo lento.

1.4 Comportamento do `this`

❌ Jeito Júnior vs ✅ Jeito Pleno

```
// ❌ JEITO JÚNIOR — perde o contexto do `this`
class ContadorJúnior {
  constructor() {
    this.valor = 0;
  }
  incrementar() {
    this.valor++;
  }
}

const c = new ContadorJúnior();
setTimeout(c.incrementar, 1000); // TypeError: Cannot read properties of undefined
```

```
// ✅ JEITO PLENO — usa arrow function (class field) para fixar o `this` léxico
class ContadorPleno {
  valor = 0;
  incrementar = () => {
    this.valor++;
    console.log('Valor atual:', this.valor);
  };
}

const c = new ContadorPleno();
setTimeout(c.incrementar, 1000); // funciona: Valor atual: 1
```

Por que a diferença: `function` declarations e métodos de classe "normais" têm seu `this` definido **dinamicamente**, no momento da chamada — depende de *quem* chamou a função. Arrow functions **não têm `this` próprio**: capturam o `this` do escopo léxico em que foram definidas. Ao passar `c.incrementar` como referência, você perde o vínculo com `c`, a menos que use arrow function como class field.

```
// node this-binding.js

const obj = {
  nome: 'Node',
  regular: function () {
    console.log('function normal:', this.nome);
  },
  arrow: () => {
    console.log('arrow function:', this?.nome);
  },
};

obj.regular();
obj.arrow();

const { regular } = obj;
try {
  regular();
} catch (e) {
  console.log('Erro ao desestruturar method:', e.message);
}
```

Retorno esperado:

```
function normal: Node
arrow function: undefined
Erro ao desestruturar method: Cannot read properties of undefined (reading 'nome')
```

💡 **DICA DE PLENO:** em entrevistas, explique **por quê**: a arrow function não cria seu próprio binding de **this** / **arguments** / **super**, então sobe na cadeia de escopos léxicos até achar um **this** definido — exatamente como aconteceria com qualquer variável comum.

Por baixo dos panos

Métodos de classe "normais" no V8 são compilados como funções que recebem `this` como um argumento implícito, resolvido em **tempo de chamada** olhando o objeto à esquerda do `.` (ou `undefined` / o objeto global em modo não-estrito, `undefined` em modo estrito — classes de ES6 são sempre **strict mode**). Já as arrow functions como class fields (`incrementar = () => {...}`) não são métodos do protótipo — são **propriedades de instância**, criadas uma nova closure por objeto dentro do **constructor**. Isso tem um custo real: cada instância carrega sua própria cópia da função (mais memória, uma alocação extra por `new`), enquanto métodos normais no protótipo são compartilhados por todas as instâncias. Em código com milhares de instâncias por segundo (ex: um `Parser` criado por requisição), isso pode ser mensurável no profiler de alocação.

Cenário de Falha em Produção

Um handler de rota Express definido como método de classe (`class PedidoController { async criar(req, res) {...} }`) registrado diretamente como `router.post('/pedidos', controller.criar)` perde o `this`. Em dev, isso quebra na primeira requisição com um erro óbvio. Mas o cenário perigoso é quando o método **não usa `this` no caminho feliz**, só numa branch de erro (ex: `this.logger.error(...)` dentro de um `catch`) — os testes e o QA passam por meses, e só em produção, na primeira exceção real, o processo lança um **`TypeError: Cannot read properties of undefined` dentro do `catch`**, mascarando o erro original e potencialmente derrubando o processo se não houver um handler de `uncaughtException`.

Dica de Produção

Esse bug é reconhecível no Sentry/APM por um padrão específico: o erro reportado é sempre **`TypeError: Cannot read properties of undefined (reading 'algumaCoisa')`** apontando para uma linha dentro de um bloco `catch`, nunca no caminho principal. Se você vir isso, a causa raiz quase sempre é perda de contexto de `this` num handler passado por referência — o stack trace do Sentry vai mostrar a exceção real engolida, porque o `TypeError` do `this` undefined substituiu ela antes de chegar no seu error handler central.

1.5 Closures aplicados a problemas reais

var vs let em loops

```
// ❌ JEITO JÚNIOR
for (var i = 0; i < 3; i++) {
  setTimeout(() => console.log('var i =', i), 10);
}
```

```
// ✅ JEITO PLENO
for (let i = 0; i < 3; i++) {
  setTimeout(() => console.log('let i =', i), 10);
}
```

Retorno esperado (executando os dois blocos em sequência):

```
var i = 3
var i = 3
var i = 3
let i = 0
let i = 1
let i = 2
```

Por que a diferença: `var` é *function-scoped* — existe uma única variável `i` compartilhada por todas as iterações. Quando os callbacks rodam, `i` já vale `3`. `let` é *block-scoped* — o motor JS cria um novo ambiente léxico a cada iteração.



Por baixo dos panos

O V8 implementa isso literalmente criando um novo *lexical environment* (um objeto interno de binding) a cada iteração do `for (let ...)` — o spec do ECMAScript instrui o motor a "copiar" o valor de `i` do ambiente anterior para o novo ambiente no início de cada iteração. Isso não é gratuito: um `for (let i...)` com closures dentro tem overhead de alocação de memória maior que `for (var i...)`, porque cada closure fecha sobre seu próprio objeto de ambiente. Para loops sem closures escapando, o V8 costuma otimizar isso via *escape analysis* e o custo desaparece — mas quando você de fato cria uma função dentro do loop (como no `setTimeout`), o compilador não pode otimizar, porque a closure pode sobreviver além do loop.

🌟 Cenário de Falha em Produção

Um padrão comum em código de processamento em lote: `for (var i = 0; i < pedidos.length; i++) { fila.adicionar(() => processarPedido(pedidos[i])); }`. Com `var`, todos os callbacks enfileirados fecham sobre a **mesma** `i`, e quando a fila finalmente processa (de forma assíncrona, minutos depois), todos processam o **último** pedido do array — silenciosamente, sem erro nenhum, só resultado de negócio errado (ex: 500 clientes recebendo notificação do pedido #500 em vez do próprio pedido).

🧠 Dica de Produção

Esse bug não aparece em teste unitário síncrono ingênuo (que costuma testar 1 item por vez) — só se manifesta com **múltiplos itens processados de forma assíncrona/atrasada**. Em produção, o sintoma é "dados duplicados ou de outro registro aparecendo nos logs/notificações", com o campo errado sempre convergindo para o **último valor do lote**. Se você ver esse padrão nos logs estruturados (mesmo `pedidoId` aparecendo em múltiplos eventos que deveriam ser de pedidos diferentes), suspeite de `var` capturado em closure dentro de loop com callback assíncrono — busque por `var` (não `let` / `const`) em loops no arquivo suspeito como primeiro passo de triagem.

Função fábrica (encapsulamento de estado sem classe)

```
// node closure-factory.js

function criarContaBancaria(saldoInicial) {
  let saldo = saldoInicial;

  return {
    depositar(valor) {
      saldo += valor;
      return saldo;
    },
    sacar(valor) {
      if (valor > saldo) throw new Error('Saldo insuficiente');
      saldo -= valor;
      return saldo;
    },
    consultarSaldo() {
      return saldo;
    },
  };
}

const conta = criarContaBancaria(100);
console.log(conta.depositar(50));
console.log(conta.sacar(30));
console.log(conta.saldo);
console.log(conta.consultarSaldo());
```

Retorno esperado:

```
150
120
undefined
120
```

💡 **DICA DE PLENO:** esse padrão (module pattern / factory function) é alternativa válida a classes com campos privados (`#saldo`) quando você quer objetos simples sem `new` , sem herança, e com encapsulamento real garantido pelo escopo léxico.

🌟 Cenário de Falha em Produção

Closures que capturam estado mutável parecem inofensivas isoladas, mas em uma aplicação com 50k usuários concorrentes, cada `criarContaBancaria` gera um novo objeto de ambiente léxico na heap — se você cria milhares dessas fábricas por segundo (ex: uma por requisição, para um cálculo temporário) e alguma referência a elas escapa (fica presa num array global de cache, num listener nunca removido, numa Promise nunca resolvida), o Garbage Collector não consegue coletar esses ambientes — cada um segura uma referência viva a `saldo` e às três funções. É o mesmo mecanismo de memory leak da seção 10.1, só que a "causa raiz" aqui é arquitetural (closures de vida longa), não um array óbvio.

🧠 Dica de Produção

Diferente de vazamento por array global (fácil de achar num heap snapshot, porque aparece como um objeto gigante), leaks por closures acumuladas aparecem no heap snapshot do Chrome DevTools como **muitas instâncias pequenas do mesmo shape/função**, sob a categoria "Closure" no comparador de snapshots — o sinal é "20 mil instâncias de uma função anônima que deveria existir só enquanto a requisição dura". Se o RSS do processo cresce de forma constante e correlacionada com volume de requisições (não com o tamanho dos dados), suspeite de closures presas por referência externa, não de um cache óbvio.

1.6 Optional Chaining (`?.`) e Nullish Coalescing (`??`)

❌ Jeito Júnior vs ✅ Jeito Pleno

```
// ❌ JEITO JÚNIOR – validações manuais aninhadas, verbosas e frágeis
function obterCidadeJunior(usuario) {
  if (usuario && usuario.endereco && usuario.endereco.cidade) {
    return usuario.endereco.cidade;
  }
  return 'Não informado';
}
```

```
// ✅ JEITO PLENO – optional chaining + nullish coalescing
function obterCidadePleno(usuario) {
  return usuario?.endereco?.cidade ?? 'Não informado';
}
```

```
// node optional-chaining-nullish.js

const usuarioCompleto = { nome: 'Ana', endereco: { cidade: 'Curitiba' } };
const usuarioSemEndereco = { nome: 'Bruno' };

function obterCidade(usuario) {
  return usuario?.endereco?.cidade ?? 'Não informado';
}

console.log(obterCidade(usuarioCompleto));
console.log(obterCidade(usuarioSemEndereco));
console.log(obterCidade(null));

// Diferença crucial entre ?? e ||
const configuracao = { limite: 0, ativo: false, nome: '' };

console.log('Com || (ERRADO para valores "falsy" válidos):');
console.log('limite:', configuracao.limite || 10); // 10 -- ERRADO, 0 é um valor válido!
console.log('ativo:', configuracao.ativo || true); // true -- ERRADO, false é válido!

console.log('Com ?? (CORRETO — só cai no default se for null ou undefined):');
console.log('limite:', configuracao.limite ?? 10); // 0 -- CORRETO
console.log('ativo:', configuracao.ativo ?? true); // false -- CORRETO
```

Retorno esperado:

```
Curitiba
Não informado
Não informado
Com || (ERRADO para valores "falsy" válidos):
limite: 10
ativo: true
Com ?? (CORRETO — só cai no default se for null ou undefined):
limite: 0
ativo: false
```

⚠️ **ARMADILHA COMUM:** usar `||` para definir valores default é um dos bugs mais comuns em código Pleno "descuidado". `||` cai no fallback para **qualquer** valor falsy (`0`, `''`, `false`, `NaN`), não apenas `null` / `undefined`. Se `0` for um valor de negócio legítimo (ex: desconto de 0%, estoque zerado), `||` vai silenciosamente substituí-lo pelo default — um bug sutil e difícil de rastrear em produção. `??` resolve isso ao checar apenas `null` / `undefined`.

🌟 Cenário de Falha em Produção

Um endpoint de checkout usa `const desconto = pedido.desconto || calcularDescontoPadrao(pedido)`. No dia da Black Friday, um cupom de "0% de desconto adicional, mas frete grátis" é aplicado — `pedido.desconto` é `0` (valor correto, calculado no carrinho), mas o `||` trata `0` como ausente e recalcula um desconto padrão, **cobrando a mais** de milhares de clientes na maior campanha do ano. O bug é invisível em teste unitário se ninguém testar explicitamente o caso "desconto igual a zero, mas presente" — é comum testar apenas "com desconto" (valor > 0) e "sem desconto" (`undefined`), pulando o caso `0`.

🧠 Dica de Produção

Esse tipo de bug de negócio não gera exceção — é **silencioso e financeiro**. Ele só aparece em produção via reconciliação (o valor cobrado no gateway de pagamento diverge do valor esperado no pedido) ou reclamação de cliente. A defesa em produção é dupla: (1) testes de propriedade (seção 5.3) que cobrem `0` explicitamente como input, e (2) um log estruturado no momento do cálculo de preço registrando **todos os componentes** (`{ subtotal, desconto, descontoOrigem: 'usuario' | 'padrao', total }`) — assim, quando o financeiro perguntar "por que esse pedido cobrou X", você consegue correlacionar pelo `pedidoId` no seu agregador de logs em segundos, em vez de tentar reproduzir o bug.

🤔 **Desafio de Mentoria:** você está revisando um PR que substitui `config.timeout || 5000` por `config.timeout ?? 5000` em uma função de configuração de retry de chamadas a um serviço externo. O autor do PR alega que é "só uma melhoria de correção, sem risco". Que pergunta você faz antes de aprovar — pensando em quem já consome essa função hoje com `timeout: 0` (querendo dizer "sem timeout") vs quem nunca configurou nada (`undefined`)? A mudança de `||` para `??` pode alterar o comportamento de produção de alguém que nunca tocou nesse código?

1.7 Debounce e Throttle

Ambos controlam a **frequência** de execução de uma função, mas com estratégias diferentes — clássico em campos de busca, scroll infinito, resize de janela, etc.

```
// node debounce-throttle.js

// DEBOUNCE: só executa após o usuário PARAR de chamar a função por X ms.
// Ideal para: campo de busca (só busca quando o usuário parou de digitar).
function debounce(fn, delayMs) {
  let timeoutId;
  return function (...args) {
    clearTimeout(timeoutId); // cancela a chamada anterior pendente
    timeoutId = setTimeout(() => fn.apply(this, args), delayMs);
  };
}

// THROTTLE: executa no máximo 1 vez a cada X ms, independente de quantas vezes for chamada.
// Ideal para: evento de scroll, resize (não pode simplesmente ignorar todas as chamadas intermediárias)
function throttle(fn, limiteMs) {
  let podeExecutar = true;
  return function (...args) {
    if (!podeExecutar) return;
    fn.apply(this, args);
    podeExecutar = false;
    setTimeout(() => (podeExecutar = true), limiteMs);
  };
}

const buscarNoServidor = debounce((termo) => {
  console.log(`[DEBOUNCE] Buscando por: "${termo}"`);
}, 300);

const registrarScroll = throttle(() => {
  console.log(`[THROTTLE] Posição de scroll registrada em ${Date.now()}`);
}, 300);

// Simulação: usuário digitando rápido "n", "no", "nod", "node" (só a última deve buscar)
['n', 'no', 'nod', 'node'].forEach((termo, i) => {
  setTimeout(() => buscarNoServidor(termo), i * 100);
});

// Simulação: 5 eventos de scroll disparados rapidamente (throttle limita a execução)
for (let i = 0; i < 5; i++) {
  setTimeout(() => registrarScroll(), i * 100);
}
```

Retorno esperado (aproximado — os tempos exatos podem variar poucos ms):

```
[THROTTLE] Posição de scroll registrada em 1690000000000
[THROTTLE] Posição de scroll registrada em 1690000000300
[DEBOUNCE] Buscando por: "node"
```

💡 **DICA DE PLENO:** debounce "atrasa e cancela" — só a última chamada dentro da janela sobrevive. Throttle "espaça" — garante uma execução periódica mesmo com chamadas contínuas. Confundir os dois é um erro clássico: usar debounce num scroll faz a UI parecer travada (só atualiza quando o usuário para de rolar); usar throttle numa busca desperdiça requisições para termos intermediários incompletos.

🌟 Cenário de Falha em Produção

No back-end, o equivalente a debounce/throttle costuma proteger recursos caros — por exemplo, um endpoint `POST /webhook/pagamento` chamado repetidamente pelo provedor de pagamento em caso de timeout de rede (retry automático do lado deles). Sem alguma forma de deduplicação (idempotência via `eventId` único + `SET NX` no Redis, não um debounce de UI), você processa o mesmo pagamento duas vezes — debitando o estoque duas vezes ou disparando dois emails de confirmação. O debounce/throttle "de front-end" não se aplica diretamente aqui, mas o **princípio** (controlar frequência/deduplicar chamadas repetidas do mesmo evento) é o mesmo, e é um erro comum de Pleno assumir que "o provedor não vai mandar o mesmo webhook duas vezes".

🧠 Dica de Produção

Para detectar processamento duplicado em produção, logue **sempre** o identificador de idempotência recebido (`eventId` , `idempotencyKey`) como campo estruturado. No APM, configure um alerta para o mesmo `eventId` aparecendo mais de uma vez dentro de uma janela curta (ex: 1 minuto) — isso é sinal de retry do provedor ou de um bug de reprocessamento na sua fila, e você quer saber disso antes que o time financeiro descubra via reconciliação bancária.

🤔 **Desafio de Mentoria:** pensando nos 6 tópicos desta seção juntos — Event Loop, microtasks, `this` , closures, `??` , debounce/throttle — qual deles você diria que tem o **maior potencial de causar um incidente silencioso em produção** (aquele que não gera erro, só resultado errado, e some sem log)? Defenda sua escolha com um cenário concreto de uma API com 50k usuários ativos.

🔗 Referências

- [Node.js Event Loop, Timers, and process.nextTick\(\)](#)
- [Jake Archibald: In The Loop \(JSConf.Asia\)](#)

- [MDN: Optional chaining \(?.\)](#)
 - [MDN: Nullish coalescing operator \(??\)](#)
 - [MDN: Closures](#)
-

2. TypeScript Avançado

2.1 Utility Types

```
// npx ts-node utility-types.ts

interface Usuario {
  id: number;
  nome: string;
  email: string;
  senha: string;
}

// Partial<T> – todas as propriedades se tornam opcionais (ótimo para PATCH/update)
type AtualizarUsuario = Partial<Usuario>;
const patch: AtualizarUsuario = { nome: 'Novo Nome' };

// Required<T> – força todas as propriedades a serem obrigatórias
interface Config {
  timeout?: number;
  retries?: number;
}
type ConfigCompleta = Required<Config>;
// const c: ConfigCompleta = {}; // ❌ erro: timeout e retries agora são obrigatórios

// Pick<T, K> – seleciona um subconjunto de propriedades
type UsuarioPublico = Pick<Usuario, 'id' | 'nome'>;
const publico: UsuarioPublico = { id: 1, nome: 'Alexandre' };

// Omit<T, K> – remove propriedades (o inverso do Pick)
type UsuarioSemSenha = Omit<Usuario, 'senha'>;
const semSenha: UsuarioSemSenha = { id: 1, nome: 'Alexandre', email: 'a@a.com' };

// Record<K, V> – cria um tipo de objeto/dicionário tipado
type Permissoes = Record<'admin' | 'editor' | 'viewer', boolean>;
const permissoes: Permissoes = { admin: true, editor: false, viewer: true };

// ReturnType<T> – extrai o tipo de retorno de uma função
function criarUsuario() {
  return { id: 1, nome: 'Ana', ativo: true };
}
type UsuarioCriado = ReturnType<typeof criarUsuario>;
const novoUsuario: UsuarioCriado = { id: 2, nome: 'Bia', ativo: false };

// Parameters<T> – extrai os tipos dos parâmetros de uma função como uma tupla
function enviarEmail(destinatario: string, assunto: string, corpo: string) {
  console.log(`Enviando para ${destinatario}: ${assunto}`);
}
type ParametrosEmail = Parameters<typeof enviarEmail>;
// equivalente a: [destinatario: string, assunto: string, corpo: string]

function enviarEmailComLog(...args: ParametrosEmail) {
  console.log('[LOG] Preparando envio de email...');
  enviarEmail(...args);
}
```

```
console.log({ patch, publico, semSenha, permissoes, novoUsuario });
enviarEmailComLog('ana@email.com', 'Bem-vinda', 'Olá, Ana!');
```

Retorno esperado:

```
{
  patch: { nome: 'Novo Nome' },
  publico: { id: 1, nome: 'Alexandre' },
  semSenha: { id: 1, nome: 'Alexandre', email: 'a@a.com' },
  permissoes: { admin: true, editor: false, viewer: true },
  novoUsuario: { id: 2, nome: 'Bia', ativo: false }
}
[LOG] Preparando envio de email...
Enviando para ana@email.com: Bem-vinda
```

⚠ **ARMADILHA COMUM:** `Omit<T, K>` não valida se `K` realmente existe em `T` da mesma forma estrita que `Pick`. Se você digitar `Omit<Usuario, 'senha'>` (com tipo), o TypeScript **não vai reclamar**. Sempre revise Omit com atenção.

💡 **DICA DE PLENO:** `Parameters<T>` é extremamente útil para criar wrappers/decorators de funções (como logging, retry, cache) sem duplicar a assinatura de tipos manualmente — o wrapper sempre fica sincronizado com a função original.

Por baixo dos panos

Isso é importante entender bem: **nenhum desses tipos existe em tempo de execução**. TypeScript é apagado (erased) pelo compilador — `Partial<Usuario>`, `Pick<Usuario, 'id'>`, tudo isso é metadado que existe só durante `tsc` (ou o transpile do `ts-node` / `esbuild` / `swc`) e desaparece completamente no JavaScript gerado. Isso significa que `Required<Config>` **não valida nada em runtime** — se um payload de API externa chegar sem `timeout`, o TypeScript não vai te salvar, porque a checagem de tipos já foi feita (e "confiada") em tempo de compilação, não quando o dado realmente chega pela rede.

🌟 Cenário de Falha em Produção

Um erro clássico de Pleno: usar `as Usuario` (type assertion) para "convencer" o TypeScript de que um `JSON.parse(body)` vindo de uma requisição HTTP é do tipo `Usuario`, sem validação real. Em produção, um cliente malicioso ou um bug no serviço consumidor manda um payload com `email` ausente. O código passa pela checagem de tipos (porque `as` desliga a checagem), a função assume que `usuario.email` existe, e o processo lança um `TypeError` não tratado ao tentar `usuario.email.toLowerCase()` — um crash em produção que o TypeScript "prometeu" que não podia acontecer.

🧠 Dica de Produção

Tipos TypeScript aparecem em **zero** lugares no seu APM ou nos seus logs — eles não existem no runtime, então não adianta procurar por eles ao debugar um incidente. O que você vai ver é o `TypeError` genérico do V8. A lição prática: para qualquer dado que atravessa uma fronteira de confiança (body HTTP, resposta de API externa, mensagem de fila), use uma biblioteca de validação de runtime (Zod, io-ts) que gera o tipo TypeScript a partir do schema de validação — assim tipo e validação nunca ficam dessincronizados, e um payload inválido vira um erro 400 controlado, não uma exceção não tratada.

2.2 Generics com constraints (`extends`)

❌ Jeito Júnior vs ✅ Jeito Pleno

```
// ❌ JEITO JÚNIOR – usa `any`, perde toda a segurança de tipos
function pegarPropriedade(obj: any, chave: any) {
  return obj[chave];
}
const nome = pegarPropriedade({ nome: 'Ana' }, 'nome'); // tipo: any
```

```
// ✅ JEITO PLENO – generic com constraint
function pegarPropriedade<T, K extends keyof T>(obj: T, chave: K): T[K] {
  return obj[chave];
}
const usuario = { nome: 'Ana', idade: 30 };
const nomeTipado = pegarPropriedade(usuario, 'nome'); // tipo inferido: string
// const erro = pegarPropriedade(usuario, 'sobrenome'); // ❌ erro em tempo de compilação
```

Por que a diferença: `K extends keyof T` restringe `K` a ser apenas uma chave existente em `T`, movendo erros de "campo errado" de runtime para compile-time.

Aplicação em classe: Repositório genérico com constraint

```
// npx ts-node generic-class.ts

interface ComId {
  id: number;
}

class RepositorioBase<T extends ComId> {
  protected itens: T[] = [];

  adicionar(item: T): void {
    this.itens.push(item);
  }

  buscarPorId(id: number): T | undefined {
    return this.itens.find((item) => item.id === id);
  }

  todos(): T[] {
    return this.itens;
  }
}

interface Produto extends ComId {
  nome: string;
  preco: number;
}

class RepositorioProdutos extends RepositorioBase<Produto> {
  buscarMaisCaroQue(valor: number): Produto[] {
    return this.itens.filter((p) => p.preco > valor);
  }
}

const repo = new RepositorioProdutos();
repo.adicionar({ id: 1, nome: 'Teclado', preco: 250 });
repo.adicionar({ id: 2, nome: 'Monitor', preco: 900 });

console.log(repo.buscarPorId(1));
console.log(repo.buscarMaisCaroQue(300));
```

Retorno esperado:

```
{ id: 1, nome: 'Teclado', preco: 250 }
[ { id: 2, nome: 'Monitor', preco: 900 } ]
```

 **DICA DE PLENO:** o `extends ComId` garante, em tempo de compilação, que **qualquer** tipo usado com `RepositorioBase<T>` tenha um `id: number`, permitindo que `buscarPorId` funcione de forma genérica e segura.

🌟 Cenário de Falha em Produção

Generics dão uma falsa sensação de segurança quando a implementação por trás usa `any` escondido. Um erro comum de Pleno "apressado": implementar

`RepositorioBase<T>.buscarPorId` delegando para uma query genérica construída via template string interpolando o nome do campo (``SELECT * FROM ${tabela} WHERE id = ${id}``) — o generic `T extends ComId` te protege da **forma** do objeto, mas não da forma como você constrói a query. Se `tabela` também vier de uma variável derivada de input do usuário (ex: nome de uma entidade dinâmica), você reintroduziu SQL Injection (seção 6.3) *dentro* de uma abstração que parecia type-safe — o generic engana o revisor de código, que assume "está tipado, está seguro".

🧠 Dica de Produção

Ao revisar (ou debugar) uma classe genérica em produção, sempre abra a implementação e pergunte "o que essa abstração genérica está *escondendo*?". No profiling de queries lentas (ex: `pg_stat_statements` no Postgres), repositórios genéricos mal implementados tendem a gerar queries menos otimizadas que código específico por entidade (porque abstrações genéricas raramente aproveitam índices específicos ou fazem `SELECT` seletivo por coluna) — se você notar uma query de forma idêntica repetida para múltiplas entidades diferentes no log lento do banco, é sinal de um repositório genérico "over-abstraído" que vale a pena especializar para as entidades de maior tráfego.

2.3 Type Guards e Narrowing

```
// npx ts-node type-guards.ts

type Quadrado = { tipo: 'quadrado'; lado: number };
type Circulo = { tipo: 'circulo'; raio: number };
type Forma = Quadrado | Circulo;

function ehQuadrado(forma: Forma): forma is Quadrado {
  return forma.tipo === 'quadrado';
}

function calcularArea(forma: Forma): number {
  if (forma.tipo === 'quadrado') {
    return forma.lado ** 2;
  }
  return Math.PI * forma.raio ** 2;
}

class ErroValidacao extends Error {}
class ErroBancoDeDados extends Error {}

function tratarErro(erro: unknown): string {
  if (erro instanceof ErroValidacao) return `Erro de validação: ${erro.message}`;
  if (erro instanceof ErroBancoDeDados) return `Erro de banco: ${erro.message}`;
  if (typeof erro === 'string') return `Erro (string): ${erro}`;
  return 'Erro desconhecido';
}

console.log(calcularArea({ tipo: 'quadrado', lado: 4 }));
console.log(calcularArea({ tipo: 'circulo', raio: 2 }));
console.log(tratarErro(new ErroValidacao('campo obrigatório')));
console.log(tratarErro('falha simples'));
```

Retorno esperado:

```
16
12.566370614359172
Erro de validação: campo obrigatório
Erro (string): falha simples
```

⚠️ ARMADILHA COMUM: usar `unknown` (em vez de `any`) para capturar erros em `catch` é a prática correta desde o TS 4.4+, mas isso **obriga** você a fazer narrowing antes de acessar propriedades — é exatamente esse atrito que evita bugs em produção.

🌟 Cenário de Falha em Produção

Um erro sutil de discriminated union: se `Forma` ganhar um terceiro membro (`Triangulo`) e `calcularArea` não for atualizada, o TypeScript **não vai reclamar** se a função usar `if/else` genérico em vez de um `switch` exaustivo — o código compila, mas silenciosamente trata triângulos como círculos (cai no `else` / `return Math.PI * forma.raio ** 2`, onde `forma.raio` é `undefined`, resultando em `NaN` propagado pelo sistema sem exceção nenhuma). Isso é conhecido como falta de **exhaustiveness checking**.

🧠 Dica de Produção

A defesa correta é terminar todo `switch` sobre union discriminada com um `default: const _exaustivo: never = forma; throw new Error(...)`. Isso faz o **compilador** (não o runtime) barrar a mudança: ao adicionar `Triangulo`, o `tsc` falha o build porque `Triangulo` não é atribuível a `never`. Em produção, se você já tem esse guard e ele nunca compilou com sucesso sem tratar o novo caso, você elimina de vez a classe de bug "novo tipo de dado não tratado silenciosamente" — que de outra forma só apareceria como `NaN` aparecendo em relatórios financeiros, dias depois, sem stack trace nenhum para rastrear.

2.4 Mapped Types e Template Literal Types

```
// npx ts-node mapped-and-template-types.ts

interface Usuario {
  nome: string;
  idade: number;
  email: string;
}

type UsuarioReadonly = { readonly [K in keyof Usuario]: Usuario[K] };
type UsuarioFormulario = { [K in keyof Usuario]?: Usuario[K] | null };

type Evento = 'criado' | 'atualizado' | 'removido';
type NomeDeHandler = `on${Capitalize<Evento>}`;

type EventMap = {
  [E in NomeDeHandler]: () => void;
};

const handlers: EventMap = {
  onCriado: () => console.log('Recurso criado'),
  onAtualizado: () => console.log('Recurso atualizado'),
  onRemovido: () => console.log('Recurso removido'),
};

const usuarioForm: UsuarioFormulario = { nome: 'Ana', idade: null };

handlers.onCriado();
handlers.onAtualizado();
console.log(usuarioForm);
```

Retorno esperado:

```
Recurso criado
Recurso atualizado
{ nome: 'Ana', idade: null }
```

 **DICA DE PLENO:** Template Literal Types combinados com Mapped Types são a base de bibliotecas como o Zod e de sistemas de rotas type-safe.

🌟 Cenário de Falha em Produção

`UsuarioReadOnly` (com `readonly` em cada campo) protege apenas em **tempo de compilação** — em runtime, `Object.freeze` não é aplicado automaticamente. Um Pleno menos experiente confia que "marquei como readonly, então está protegido" e passa esse objeto para uma função de terceiros (biblioteca externa, código legado sem tipos) que faz `objeto.nome = 'outra coisa'` diretamente — o JavaScript compilado permite a mutação sem nenhum aviso, porque `readonly` já foi apagado. Se esse objeto for compartilhado entre requisições concorrentes (ex: um objeto de configuração cacheado em módulo, reaproveitado a cada request), você tem uma condição de corrida sutil onde a requisição de um usuário muta um objeto que outro usuário concorrente também está lendo.

🧠 Dica de Produção

Esse tipo de mutação inesperada se manifesta como "dado de um usuário aparecendo na resposta de outro usuário" — um dos bugs mais graves e assustadores de diagnosticar, porque parece intermitente e ligado a timing/carga (só acontece sob concorrência real, nunca em teste local com uma requisição por vez). Se isso acontecer, adicione `Object.freeze()` real no objeto compartilhado (não confie só no tipo) e, para investigar depois do fato, correlacione por `requestId` nos logs estruturados — se dois `requestId` diferentes referenciam o mesmo objeto de dados no mesmo timestamp, você achou a fonte do compartilhamento indevido.

2.5 O operador `satisfies` (TypeScript 4.9+)

O problema clássico: anotar um tipo explícito (`: TipoX`) **restringe** o valor à interface do tipo (perdendo inferência mais específica), enquanto não anotar nada **perde a checagem** de que o objeto está correto. `satisfies` resolve os dois problemas ao mesmo tempo.

❌ Jeito Júnior vs ✅ Jeito Pleno

```
// npx ts-node satisfies-operator.ts

type Cor = 'vermelho' | 'verde' | 'azul';

// ❌ JEITO JÚNIOR – anotação de tipo explícita "achata" a inferência
const paletaAnotada: Record<Cor, string> = {
  vermelho: '#FF0000',
  verde: '#00FF00',
  azul: '#0000FF',
};

// paletaAnotada.vermelho tem tipo `string` genérico -- perdemos a informação
// de que é ESPECIFICAMENTE '#FF0000' (útil para autocomplete/literal types)

// ❌ JEITO JÚNIOR (alternativa) – sem anotação nenhuma, sem checagem de erro
const paletaSemChecagem = {
  vermelho: '#FF0000',
  verde: '#00FF00',
  azul: '#0000FF', // typo! TypeScript NÃO avisa, pois não há contrato nenhum
};
```

```
// ✅ JEITO PLENO – satisfies: valida a forma E mantém a inferência literal
const paletaPleno = {
  vermelho: '#FF0000',
  verde: '#00FF00',
  azul: '#0000FF',
} satisfies Record<Cor, string>;

// paletaPleno.vermelho tem tipo LITERAL '#FF0000' (não apenas `string`)
console.log(paletaPleno.vermelho.toUpperCase()); // autocomplete completo, sem perder o literal

// Se houvesse um typo aqui, o TypeScript acusaria erro IMEDIATAMENTE:
// const paletaComErro = {
//   vermelho: '#FF0000',
//   verde: '#00FF00',
//   azull: '#0000FF', // ❌ Object literal may only specify known properties
// } satisfies Record<Cor, string>;

console.log(paletaPleno);
```

Retorno esperado:

```
#FF0000
{ vermelho: '#FF0000', verde: '#00FF00', azul: '#0000FF' }
```

💡 **DICA DE PLENO:** use `satisfies` quando quiser **validar a forma de um objeto contra um tipo/contrato**, mas ainda precisar do **tipo mais específico possível** para o valor resultante (essencial em configs, mapas de constantes, e ao trabalhar com bibliotecas que dependem de literal types, como builders de rotas ou temas de UI).

🧠 Dica de Produção

`satisfies` é uma ferramenta de **compile-time** — não existe rastro dela em produção, nos logs, ou no APM. O ganho real para o time é indireto: menos bugs de "typo em chave de config" chegando a produção, porque eles são pegos no CI (`tsc --noEmit` no pipeline) antes do deploy. Se seu time ainda vê erros de "propriedade não existe" ou "chave duplicada" só depois do deploy, é sinal de que o build de CI não está rodando `tsc` com o mesmo rigor do editor — vale auditar o `tsconfig.json` usado no pipeline versus o usado localmente.

🤔 **Desafio de Mentoria:** o TypeScript te dá confiança em tempo de compilação, mas **zero** garantia em runtime — tipos são apagados. Dado isso, num serviço que recebe webhooks de um parceiro externo (dado 100% fora do seu controle), onde exatamente na sua arquitetura você colocaria a "fronteira de confiança" entre "dado não confiável, precisa de validação de runtime (Zod/io-ts)" e "dado já confiável, pode usar apenas tipos estáticos do TypeScript"? E o que muda nessa resposta se o parceiro externo também for uma equipe interna da mesma empresa, mas em outro serviço?

🔗 Referências

- [TypeScript Handbook — Utility Types](#)
- [TypeScript Handbook — Generics](#)
- [TypeScript Handbook — Narrowing](#)
- [TypeScript 4.9 Release Notes — The `satisfies` Operator](#)
- [Type Challenges \(exercícios avançados de tipos\)](#)

3. Node.js Internals

3.1 Streams: Readable, Writable, Transform e Pipeline

Streams processam dados em **pedaços (chunks)**, sem carregar o arquivo inteiro na memória — essencial para arquivos grandes.

```
// node streams-csv-para-json.js
const { Transform, pipeline } = require('stream');
const fs = require('fs');
const path = require('path');

const csvPath = path.join(__dirname, 'dados.csv');
fs.writeFileSync(csvPath, 'nome,idade\nAna,30\nBruno,25\nCarla,40\n');

let cabecalho = null;
let buffer = '';

const csvParaJson = new Transform({
  transform(chunk, _encoding, callback) {
    buffer += chunk.toString();
    const linhas = buffer.split('\n');
    buffer = linhas.pop();

    for (const linha of linhas) {
      if (!linha.trim()) continue;
      const valores = linha.split(',');

      if (!cabecalho) {
        cabecalho = valores;
        continue;
      }

      const obj = Object.fromEntries(cabecalho.map((h, i) => [h, valores[i]]));
      this.push(JSON.stringify(obj) + '\n');
    }
    callback();
  },
});

const origem = fs.createReadStream(csvPath, { encoding: 'utf8' });
const destino = fs.createWriteStream(path.join(__dirname, 'dados.jsonl'));

pipeline(origem, csvParaJson, destino, (erro) => {
  if (erro) {
    console.error('Pipeline falhou:', erro);
    return;
  }
  console.log('Pipeline concluído com sucesso.');
  console.log(fs.readFileSync(path.join(__dirname, 'dados.jsonl'), 'utf8'));
});
```

Retorno esperado:

```
Pipeline concluído com sucesso.
{"nome":"Ana","idade":"30"}
{"nome":"Bruno","idade":"25"}
{"nome":"Carla","idade":"40"}
```

pipeline() vs .pipe() manual

```
// ❌ JEITO JÚNIOR — encadeamento manual, NÃO propaga erros nem faz cleanup
fs.createReadStream('origem.txt')
  .pipe(algunTransform)
  .pipe(fs.createWriteStream('destino.txt'));
// Se algumTransform emitir 'error', os streams anteriores/posteriores
// ficam "pendurados" — sem destroy(), vazando file descriptors.
```

```
// ✅ JEITO PLENO — pipeline cuida de erros e cleanup automaticamente
const { pipeline } = require('stream/promises');

async function processar() {
  try {
    await pipeline(
      fs.createReadStream('origem.txt'),
      algumTransform,
      fs.createWriteStream('destino.txt')
    );
    console.log('Concluído com sucesso');
  } catch (erro) {
    console.error('Pipeline falhou, streams já foram limpos:', erro.message);
  }
}
```

⚠️ **ARMADILHA COMUM:** usar `.pipe()` encadeado manualmente **não propaga erros automaticamente** — sempre prefira `pipeline()` (ou `stream/promises`), que trata erros e faz cleanup automaticamente.

 Por baixo dos panos

Streams em Node existem porque a libuv opera em **buffers de tamanho fixo** por natureza de I/O do SO — ler um arquivo de 2GB de uma vez exigiria alocar 2GB contíguos no heap do V8 (que tem limite padrão de ~2-4GB dependendo da versão/arquitetura via `--max-old-space-size`). Cada `read()` de uma stream dispara uma chamada de sistema que devolve um chunk (tipicamente 64KB por padrão para arquivos), e o **backpressure** é o mecanismo que impede o produtor de encher a memória do consumidor: quando `write()` retorna `false`, significa que o buffer interno do stream de destino está cheio, e o produtor deve esperar o evento `'drain'` antes de continuar escrevendo. `pipeline()` gerencia esse backpressure automaticamente — é o motivo pelo qual você raramente precisa pensar nisso manualmente.

🌟 Cenário de Falha em Produção

Um endpoint de exportação de relatório em CSV usa `.pipe()` manual (não `pipeline()`) entre uma query de banco (via cursor/stream) e a resposta HTTP. Se o cliente fecha a conexão no meio do download (aba fechada, timeout do lado do cliente), o stream de resposta (`res`) é destruído, mas sem `pipeline()` cuidando do erro, o stream de leitura do banco **continua aberto**, mantendo o cursor da query ativo. Sob carga (múltiplos usuários exportando relatórios e fechando a aba cedo), você acumula cursores de banco abandonados até esgotar o pool de conexões — um **sintoma** que parece "banco lento" mas é, na verdade, vazamento de streams não fechados.

🧠 Dica de Produção

Monitore o número de conexões ativas no pool do banco (a maioria dos clients/ORMs expõe isso: `pool.totalCount`, `pool.idleCount`) como métrica customizada no Prometheus. Um crescimento constante de conexões "em uso" que nunca volta a `idle`, especialmente correlacionado com picos de exportação/relatórios, é a assinatura de streams não finalizados corretamente. `AbortController` propagado via `pipeline(..., { signal })` é a forma correta de garantir que o cancelamento do lado do cliente propague para o cursor do banco.

3.2 Buffers

```
// node buffers.js

const buf1 = Buffer.from('Olá, Node!', 'utf8');
const buf2 = Buffer.alloc(10);
const buf3 = Buffer.from([72, 101, 108, 108, 111]);

console.log('buf1 (hex):', buf1.toString('hex'));
console.log('buf1 (base64):', buf1.toString('base64'));
console.log('buf1 (utf8):', buf1.toString('utf8'));
console.log('buf3 (utf8):', buf3.toString('utf8'));
console.log('buf2:', buf2);
console.log('tamanho de buf1 em bytes:', buf1.length);

console.log('"" .length (JS string):', ''.length);
console.log('Buffer.from("").length:', Buffer.from('').length);
```

Retorno esperado:

```

buf1 (hex): 4fc3a16c c3a1... (hex contínuo – ilustrativo)
buf1 (base64): T2zDoSwgTm9kZSE=
buf1 (utf8): Olá, Node!
buf3 (utf8): Hello
buf2: <Buffer 00 00 00 00 00 00 00 00 00 00>
tamanho de buf1 em bytes: 11
"é".length (JS string): 1
Buffer.from("é").length: 2

```

 **DICA DE PLENO:** o clássico bug de "contador de caracteres errado com acentos" vem da confusão entre `.length` de `string` (unidades UTF-16) e `.length` de `Buffer` (bytes reais em UTF-8). Ao validar tamanho de campos vindos de upload/rede, pense em qual das duas contagens você realmente precisa.

Cenário de Falha em Produção

Um campo de "biografia" limitado a "500 caracteres" no front-end é validado no back-end com `if (bio.length > 500) throw new Error(...)`. Um usuário brasileiro escreve uma bio cheia de acentos e emojis; `bio.length` (UTF-16) conta 480 "unidades", passa na validação — mas ao persistir num banco com coluna `VARCHAR(500)` configurada para contar **bytes** (comum em MySQL com certas collations), o INSERT falha com um erro de truncamento, ou pior, o banco trunca silenciosamente o campo no meio de um caractere multi-byte, corrompendo o texto salvo (efeito conhecido como "mojibake").

Dica de Produção

Esse bug tem uma assinatura geográfica muito clara: ele **só acontece para usuários com nomes/textos em português, com acento, emoji, ou caracteres não-ASCII** — se seu time de suporte reportar "alguns usuários específicos têm erro ao salvar perfil e outros não", e não houver padrão óbvio de dados, verifique se o campo problemático tem caracteres multi-byte. No log de erro do banco, procure por mensagens de "Data too long for column" ocorrendo só para uma fração dos usuários — isso é a pista. A correção correta é validar com `Buffer.byteLength(texto, 'utf8')` quando o limite do lado do banco é em bytes, não em `.length` de string.

3.3 Worker Threads

```
// node worker-fibonacci.js
const { Worker, isMainThread, parentPort, workerData } = require('worker_threads');

function fibonacci(n) {
  if (n <= 1) return n;
  return fibonacci(n - 1) + fibonacci(n - 2);
}

if (isMainThread) {
  console.log('Thread principal iniciada. Disparando worker para Fibonacci(40)...');
  const inicio = Date.now();
  const worker = new Worker(__filename, { workerData: { n: 40 } });

  worker.on('message', (resultado) => {
    console.log(`Resultado do Worker: ${resultado} (levou ${Date.now() - inicio}ms)`);
  });
  worker.on('error', (erro) => console.error('Erro no worker:', erro));

  let contador = 0;
  const intervalo = setInterval(() => {
    console.log(`Main thread continua livre... tick ${++contador}`);
    if (contador === 3) clearInterval(intervalo);
  }, 50);
} else {
  const resultado = fibonacci(workerData.n);
  parentPort.postMessage(resultado);
}
```

Retorno esperado (a ordem exata dos "ticks" pode variar, mas surgem ANTES do resultado):

```
Thread principal iniciada. Disparando worker para Fibonacci(40)...
Main thread continua livre... tick 1
Main thread continua livre... tick 2
Main thread continua livre... tick 3
Resultado do Worker: 102334155 (levou 850ms)
```

⚠️ **ARMADILHA COMUM:** rodar `fibonacci(40)` diretamente na thread principal bloquearia completamente o servidor. Worker Threads resolvem isso, ao custo de comunicação via serialização de mensagens (sem memória compartilhada por padrão, exceto com `SharedArrayBuffer`).

Quando usar: cálculos matemáticos pesados, processamento de imagem/vídeo, compressão, criptografia intensiva. **Quando NÃO usar:** operações de I/O (já são assíncronas por natureza)

— criar um worker para elas só adiciona overhead.

Por baixo dos panos

Cada **Worker** é uma instância **completa e separada** do V8 e da libuv rodando numa thread do SO própria — não compartilha heap de JS com a thread principal por padrão. A comunicação via **postMessage** usa o **algoritmo de clonagem estruturada** (o mesmo do **structuredClone**, seção 10.3): os dados são serializados, copiados para a outra thread, e desserializados — isso tem custo de CPU e memória proporcional ao tamanho do dado trocado. Para dados grandes e mutáveis compartilhados entre threads sem esse custo de cópia, existe **SharedArrayBuffer** (memória real compartilhada, com os riscos clássicos de concorrência: race conditions, necessidade de **Atomics** para sincronização).

Cenário de Falha em Produção

Uma startup decide mover todo processamento de imagem (resize de avatar de usuário) para Worker Threads para "não bloquear o event loop". Sob carga de Black Friday, criam um novo **Worker por requisição** em vez de manter um pool fixo. Cada criação de Worker tem overhead de inicialização (~10-50ms, uma nova instância de V8 subindo) — com 50k usuários fazendo upload simultâneo, o processo gasta mais tempo criando/destruindo workers do que processando imagens, e o consumo de memória RSS explode porque cada Worker carrega sua própria cópia do V8 heap base (dezenas de MB cada, independente do trabalho realizado).

Dica de Produção

Monitore `process.memoryUsage().rss` e o número de threads do processo Node (`ps -eLf | grep node | wc -l` no Linux, ou via **htop** em modo thread) — um crescimento de RSS correlacionado linearmente com o número de requisições concorrentes (não com o tamanho dos dados processados) é sinal de workers sendo criados sem pool/reuso. A correção é usar um **pool de workers de tamanho fixo** (bibliotecas como **piscina** ou **workerpool** resolvem isso prontas), dimensionado para `os.cpus().length`, reaproveitando as mesmas threads entre requisições.

3.4 Child Processes: `exec`, `spawn`, `fork`

```
// node child-processes.js
const { exec, spawn, fork } = require('child_process');

exec('echo "Resultado via exec"', (erro, stdout) => {
  console.log('[exec]', stdout.trim());
});

const processoSpawn = spawn('node', ['-e', 'console.log("Resultado via spawn")']);
processoSpawn.stdout.on('data', (data) => {
  console.log('[spawn]', data.toString().trim());
});
```

```
// worker-fork.js (arquivo separado)
process.on('message', (msg) => {
  process.send(`Processado: ${msg.toUpperCase()}`);
});
```

```
// no arquivo principal
const filho = fork('./worker-fork.js');
filho.send('ola do processo pai');
filho.on('message', (resposta) => {
  console.log('[fork]', resposta);
  filho.kill();
});
```

Retorno esperado (ordem pode variar por ser assíncrono):

```
[exec] Resultado via exec
[spawn] Resultado via spawn
[fork] Processado: OLA DO PROCESSO PAI
```

Tabela de decisão:

Método	Shell?	Buffer ou Stream?	Uso típico
<code>exec</code>	Sim	Buffer (limite ~1MB)	comandos curtos, ex: <code>git status</code>
<code>spawn</code>	Não (por padrão)	Stream	comandos com saída grande/contínua, ex: <code>ffmpeg</code>
<code>fork</code>	N/A (só Node)	Stream + IPC estruturado	dividir trabalho entre processos Node com troca de mensagens

⚠️ **ARMADILHA COMUM:** usar `exec` com input vindo do usuário abre porta para **command injection**. Se precisar de argumentos dinâmicos, use `spawn` (ou `execFile`), passando argumentos como `array`, nunca concatenando string.

💣 Cenário de Falha em Produção

`exec` monta um comando via `shell` (`/bin/sh -c`), então argumentos concatenados diretamente na string são interpretados pelo shell antes de chegarem ao programa. Se um serviço de conversão de arquivos aceita um nome de arquivo do usuário e roda `exec(\ convert ${nomeArquivo} saida.png`)`, um nome de arquivo malicioso como `"; rm -rf /app/uploads; echo"` executa comandos arbitrários no servidor — não é um caso hipotético, é o padrão real de vulnerabilidades de command injection encontradas em auditorias de segurança de APIs Node.

🧠 Dica de Produção

`spawn` / `execFile` com argumentos como array **não passam pelo shell** — o array vira `argv` diretamente para a syscall `execve`, então caracteres especiais de shell (`;`, `|`, `&&`, ```) são tratados como texto literal, não como sintaxe. Em produção, se você tiver ferramentas de segurança (SAST, como Semgrep ou CodeQL) rodando no CI, elas tipicamente sinalizam qualquer uso de `exec` / `execSync` com template strings interpoladas — trate esse alerta como bloqueante, nunca como falso positivo sem investigar a fundo primeiro.

3.5 `Promise.allSettled`

Diferente de `Promise.all` (que rejeita tudo assim que **uma** promessa falha), `Promise.allSettled` espera **todas** terminarem — sucesso ou falha — e retorna o resultado de cada uma individualmente. Essencial quando você precisa processar um lote de operações independentes e não pode deixar uma falha derrubar as demais.

```
// node promise-allsettled.js

async function buscarUsuario(id) {
  if (id === 2) throw new Error(`Usuário ${id} não encontrado`);
  await new Promise((r) => setTimeout(r, 50));
  return { id, nome: `Usuário ${id}` };
}

async function main() {
  const ids = [1, 2, 3];
  const resultados = await Promise.allSettled(ids.map((id) => buscarUsuario(id)));

  const sucessos = resultados
    .filter((r) => r.status === 'fulfilled')
    .map((r) => r.value);

  const falhas = resultados
    .filter((r) => r.status === 'rejected')
    .map((r) => r.reason.message);

  console.log('Resultados brutos:', resultados);
  console.log('Sucessos:', sucessos);
  console.log('Falhas:', falhas);
}

main();
```

Retorno esperado:

```
Resultados brutos: [
  { status: 'fulfilled', value: { id: 1, nome: 'Usuário 1' } },
  { status: 'rejected', reason: Error: Usuário 2 não encontrado },
  { status: 'fulfilled', value: { id: 3, nome: 'Usuário 3' } }
]
Sucessos: [ { id: 1, nome: 'Usuário 1' }, { id: 3, nome: 'Usuário 3' } ]
Falhas: [ 'Usuário 2 não encontrado' ]
```

💡 **DICA DE PLENO:** use **Promise.all** quando **todas** as operações precisam ter sucesso para o resultado fazer sentido (ex: montar um objeto agregando várias fontes obrigatórias). Use **Promise.allSettled** quando as operações são **independentes** e uma falha isolada não deve invalidar as demais (ex: notificar 100 usuários — se 3 falharem, os outros 97 continuam recebendo).

🌟 Cenário de Falha em Produção

Trocar `Promise.all` por `Promise.allSettled` sem revisar o código consumidor é um erro clássico de "correção apressada" depois de um incidente de "uma falha derrubou tudo". Se o código que consome o resultado não filtra `status === 'rejected'` corretamente (ex: esquece de tratar as falhas e só itera sobre os valores assumindo que todos têm `.value`), você troca um crash óbvio (`Promise.all` rejeitando, erro visível) por um **silenciamento de falhas** — 3 em cada 100 notificações falham silenciosamente, ninguém percebe, e o time só descobre semanas depois que uma feature crítica (ex: reset de senha) tem uma taxa de falha de 3% nunca investigada.

🧠 Dica de Produção

Sempre que usar `Promise.allSettled`, logue explicitamente a contagem de falhas como métrica (`contadorFalhasNotificacao.inc(falhas.length)`) — nunca deixe `allSettled` "engolir" erros silenciosamente só porque o código não quebrou. No Datadog/Grafana, um painel simples de "taxa de rejeição em operações em lote" (`falhas / total`) evita que degradações parciais fiquem invisíveis atrás de uma API que "sempre responde 200 OK".

🤔 **Desafio de Mentoria:** dos cinco tópicos desta seção — Streams, Buffers, Worker Threads, Child Processes, `Promise.allSettled` — qual você usaria para resolver o seguinte problema: sua API precisa gerar um PDF de 200 páginas a partir de dados de um pedido, um processo CPU-intensivo que hoje trava a thread principal por ~2 segundos, degradando a latência de **todas** as outras rotas durante esse tempo? Justifique por que as outras opções da lista **não** resolveriam esse problema específico.

🔗 Referências

- [Node.js Streams API \(docs\)](#)
- [Node.js Buffer \(docs\)](#)
- [Node.js Worker Threads \(docs\)](#)
- [Node.js Child Process \(docs\)](#)
- [MDN: Promise.allSettled\(\)](#)

4. Padrões de Código Pleno/Sênior

4.1 Repository Pattern

```
// npm install @prisma/client
// npx ts-node repository-pattern.ts

interface Usuario {
  id: number;
  nome: string;
  email: string;
}

interface UsuarioRepository {
  buscarPorId(id: number): Promise<Usuario | null>;
  criar(dados: Omit<Usuario, 'id'>): Promise<Usuario>;
}

class PrismaUsuarioRepository implements UsuarioRepository {
  constructor(private prisma: any) {}

  async buscarPorId(id: number): Promise<Usuario | null> {
    return this.prisma.usuario.findUnique({ where: { id } });
  }

  async criar(dados: Omit<Usuario, 'id'>): Promise<Usuario> {
    return this.prisma.usuario.create({ data: dados });
  }
}

class UsuarioRepositoryEmMemoria implements UsuarioRepository {
  private usuarios: Usuario[] = [];
  private proximoId = 1;

  async buscarPorId(id: number): Promise<Usuario | null> {
    return this.usuarios.find((u) => u.id === id) ?? null;
  }

  async criar(dados: Omit<Usuario, 'id'>): Promise<Usuario> {
    const novo = { id: this.proximoId++, ...dados };
    this.usuarios.push(novo);
    return novo;
  }
}

class UsuarioService {
  constructor(private repo: UsuarioRepository) {}

  async registrar(nome: string, email: string): Promise<Usuario> {
    if (!email.includes('@')) throw new Error('Email inválido');
    return this.repo.criar({ nome, email });
  }
}

async function main() {
  const service = new UsuarioService(new UsuarioRepositoryEmMemoria());
  const usuario = await service.registrar('Alexandre', 'alexandre@email.com');
```

```
console.log(usuario);
}

main();
```

Retorno esperado:

```
{ id: 1, nome: 'Alexandre', email: 'alexandre@email.com' }
```

💡 **DICA DE PLENO:** o ganho real do Repository Pattern é **testabilidade**: seus testes de **UsuarioService** rodam em milissegundos, sem subir container de banco.

Por baixo dos panos

O Repository Pattern não muda nada em runtime — é puramente uma questão de **onde a dependência é resolvida**. Sem ele, **UsuarioService** faz **new PrismaClient()** internamente (acoplamento direto); com ele, a instância concreta é injetada de fora. A diferença real acontece na hora de testar: sem abstração, seu teste "unitário" na verdade abre uma conexão TCP real com o Postgres, espera round-trips de rede, e depende do estado do banco — não é mais um teste unitário, é um teste de integração disfarçado, e a suíte inteira fica lenta (segundos em vez de milissegundos) conforme o projeto cresce.

💥 Cenário de Falha em Produção

Um erro comum de Pleno: implementar o **RepositorioBase** (ou **PrismaUsuarioRepository**) de forma "genérica demais", escondendo detalhes de performance importantes atrás da interface. Por exemplo, **buscarPorId** que internamente faz **findMany()** e filtra em memória "porque é mais simples", funcionando bem com 100 registros em dev, mas em produção com 2 milhões de usuários, essa implementação "esconde" um full table scan atrás de uma assinatura de método inocente — o code reviewer, vendo só a interface **Promise<Usuario | null>**, não tem como saber que aquilo é uma bomba de performance sem abrir a implementação.

 Dica de Produção

Abstrações do tipo Repository tendem a **esconder** custos de I/O do olho nu — é fácil, ao ler `await this.repo.buscarPorId(id)`, esquecer que aquilo pode ser uma query lenta. No APM, sempre instrumente as chamadas ao repositório com spans nomeados (a maioria dos ORMs modernos, como Prisma, já integra automaticamente com Datadog/New Relic via middleware) para que uma query lenta apareça no trace da requisição com nome e SQL gerado, não apenas como "tempo perdido" genérico dentro do handler HTTP. Sem isso, você vê a rota lenta no APM, mas não *onde* dentro dela o tempo foi gasto.

4.2 Tratamento de Erros: Wrapper assíncrono + Exception Filter

❌ Jeito Júnior vs ✅ Jeito Pleno

```
// ❌ JEITO JÚNIOR - try/catch repetido em toda rota
app.get('/usuarios/:id', async (req, res) => {
  try {
    const usuario = await buscarUsuario(req.params.id);
    res.json(usuario);
  } catch (erro) {
    res.status(500).json({ erro: erro.message });
  }
});
```

```
// ✅ JEITO PLENO – wrapper assíncrono + exception filter centralizado
// npm install express
// npx ts-node error-handling.ts

import express, { Request, Response, NextFunction, RequestHandler } from 'express';

class ErroNaoEncontrado extends Error {
  status = 404;
}

class ErroValidacao extends Error {
  status = 400;
}

function asyncHandler(fn: RequestHandler): RequestHandler {
  return (req, res, next) => {
    Promise.resolve(fn(req, res, next)).catch(next);
  };
}

const app = express();
app.use(express.json());

async function buscarUsuario(id: string) {
  if (id !== '1') throw new ErroNaoEncontrado(`Usuário ${id} não existe`);
  return { id: 1, nome: 'Ana' };
}

app.get(
  '/usuarios/:id',
  asyncHandler(async (req: Request, res: Response) => {
    const usuario = await buscarUsuario(req.params.id);
    res.json(usuario);
  })
);

app.use((erro: any, req: Request, res: Response, next: NextFunction) => {
  const status = erro.status ?? 500;
  console.error(`[${status}]`, erro.message);
  res.status(status).json({ erro: erro.message, tipo: erro.constructor.name });
});

app.listen(3000, () => console.log('Servidor rodando na porta 3000'));
```

Retorno esperado ao acessar **GET /usuarios/2** :

```
{ "erro": "Usuário 2 não existe", "tipo": "ErroNaoEncontrado" }
```

(status HTTP 404; no terminal: **[404] Usuário 2 não existe**)

Por que a diferença: sem o wrapper, um **throw** numa rota **async** não é capturado automaticamente pelo Express (antes da v5) — a Promise rejeitada vira uma unhandled rejection

silenciosa. O wrapper encaminha qualquer erro para `next(erro)`, que cai no middleware de erro central.

Por baixo dos panos

Uma `unhandled rejection` não é um erro comum — o Node trata isso como um evento de processo (`process.on('unhandledRejection', ...)`), e desde o Node 15, o comportamento **padrão** (sem handler customizado) é derrubar o processo inteiro, equivalente a uma exceção não capturada. Isso significa que, sem o `asyncHandler`, uma única rota mal tratada pode, sob certas condições de erro, **derrubar todo o servidor** — matando todas as outras requisições em andamento dos outros 49.999 usuários simultâneos, não só a que causou o erro.

Cenário de Falha em Produção

Um time migra parte das rotas para o padrão `asyncHandler`, mas esquece uma rota legada (ou um novo desenvolvedor cria uma rota nova sem saber da convenção do time, porque não há lint enforcing isso). Em condições normais, a rota funciona bem — o erro só ocorre num caso raro de borda (ex: um timeout de rede para um serviço externo, 1 em cada 10 mil requisições). Quando esse caso raro finalmente acontece em produção às 3h da manhã, o processo inteiro derruba (`process.exit` implícito por `unhandled rejection`), o Kubernetes/PM2 reinicia o container, e por alguns segundos **toda** a API fica fora do ar — não só a rota com bug.

Dica de Produção

Configure sempre um handler global de `unhandledRejection` e `uncaughtException` que loga o erro completo com stack trace **antes** de derrubar o processo intencionalmente (`process.exit(1)`) — nunca tente "continuar" depois de uma dessas, o estado do processo é considerado não confiável. No seu APM, configure um alerta de **restart de processo/pod** (não só de taxa de erro HTTP) — reinícios inesperados e frequentes de um mesmo serviço, sem deploy correspondente, são o sinal mais forte de rotas sem tratamento de erro adequado. Adicione também um linter (ESLint com regra customizada, ou revisão de PR) que bloqueie `async (req, res) => {}` direto em rotas sem passar pelo wrapper — prevenção em tempo de code review é mais barata que debug em produção.

4.3 Cache Aside com Redis

```
// npm install ioredis
// npx ts-node cache-aside.ts

import Redis from 'ioredis';
const redis = new Redis();

interface Produto {
  id: number;
  nome: string;
  preco: number;
}

async function buscarProdutoNoBanco(id: number): Promise<Produto> {
  console.log(`[BANCO] Consultando produto ${id}... (operação lenta)`);
  await new Promise((r) => setTimeout(r, 300));
  return { id, nome: 'Teclado Mecânico', preco: 350 };
}

const TTL_SEGUNDOS = 60;

async function buscarProdutoComCache(id: number): Promise<Produto> {
  const chave = `produto:${id}`;
  const emCache = await redis.get(chave);
  if (emCache) {
    console.log('[CACHE] HIT');
    return JSON.parse(emCache);
  }
  console.log('[CACHE] MISS');
  const produto = await buscarProdutoNoBanco(id);
  await redis.set(chave, JSON.stringify(produto), 'EX', TTL_SEGUNDOS);
  return produto;
}

async function atualizarPrecoProduto(id: number, novoPreco: number) {
  await redis.del(`produto:${id}`);
  console.log(`[CACHE] Chave produto:${id} invalidada`);
}

async function main() {
  await buscarProdutoComCache(1);
  await buscarProdutoComCache(1);
  await atualizarPrecoProduto(1, 400);
  await buscarProdutoComCache(1);
  process.exit(0);
}

main();
```

Retorno esperado:

```
[CACHE] MISS
[BANCO] Consultando produto 1... (operação lenta)
[CACHE] HIT
[CACHE] Chave produto:1 invalidada
[CACHE] MISS
[BANCO] Consultando produto 1... (operação lenta)
```

⚠ **ARMADILHA COMUM:** cachear sem TTL "porque vou invalidar manualmente sempre" é uma promessa que vai ser quebrada em algum código futuro. Sempre defina TTL como rede de segurança.

💡 **DICA DE PLENO:** cuidado com **cache stampede** — se uma chave popular expira e 1000 requisições chegam simultaneamente, todas dão MISS e martelam o banco ao mesmo tempo. Padrões avançados usam locks distribuídos (**SET NX**) ou TTL antecipado para mitigar isso.

Por baixo dos panos

O código de **buscarProdutoComCache** como está escrito tem uma janela de corrida (*race window*) entre o **await redis.get(chave)** (MISS) e o **await redis.set(chave, ...)** : qualquer requisição que chegue **durante** essa janela também vai dar MISS, porque a chave ainda não foi escrita. Como o Node processa requisições concorrentemente (múltiplas requisições HTTP podem estar "no meio" de suas próprias execuções ao mesmo tempo, intercaladas pelo event loop, mesmo sendo single-threaded), 1000 requisições chegando no mesmo milissegundo após a expiração de uma chave popular podem **todas** passar pelo **if (emCache)** como falso antes que qualquer uma delas termine de escrever no Redis — cada uma dispara sua própria chamada a **buscarProdutoNoBanco** , multiplicando a carga no banco por 1000x exatamente no pior momento possível (chave popular = alto tráfego).

🌟 Cenário de Falha em Produção — respondendo diretamente ao cenário proposto

"O que acontece se o Redis cair enquanto `buscarProdutoComCache` está executando?" — Depende de como o cliente Redis (`ioredis`) está configurado. Por padrão, `ioredis` tenta reconectar automaticamente, mas enquanto a conexão está fora, cada `await redis.get(chave)` rejeita com um erro de conexão. Se o código não tiver `try/catch` ao redor da chamada ao Redis (como no exemplo do material, que assume Redis sempre disponível), essa rejeição **propaga como erro da função inteira** — `buscarProdutoComCache` lança exceção, e se não houver fallback, a rota inteira retorna 500 para o usuário, mesmo que o banco de dados (a fonte real dos dados) esteja perfeitamente saudável. Você transformou uma otimização de performance (cache) em um **ponto único de falha** — o sistema fica *menos* disponível com cache do que sem, porque agora depende de dois sistemas (banco E Redis) em vez de um.

Como evitar Cache Stampede: três técnicas complementares, em ordem crescente de sofisticação:

1. **Lock distribuído (`SET chave:lock NX EX 5`):** a primeira requisição que der MISS tenta adquirir um lock; se conseguir, ela busca no banco e popula o cache; as demais, vendo o lock já adquirido, **esperam um pouco e tentam ler o cache de novo** (polling curto) em vez de irem direto ao banco.
2. **TTL antecipado (early expiration):** guarde no valor cacheado um timestamp de "expiração lógica" menor que o TTL real do Redis. Quando uma requisição percebe que a expiração lógica já passou (mas o TTL do Redis ainda não), ela serve o valor **ligeiramente stale** enquanto dispara, em background, uma única atualização — as demais requisições concorrentes continuam recebendo o valor antigo sem saber que uma atualização está em curso.
3. **Fallback gracioso para falha do Redis:** envolva a chamada ao Redis em `try/catch` — se o Redis estiver fora, **degrade** para buscar direto no banco (mais lento, mas funcional) em vez de propagar o erro. Isso é literalmente o princípio do *circuit breaker* aplicado ao cache: o cache é uma otimização, nunca uma dependência obrigatória para a rota funcionar.

 Dica de Produção

Cache stampede tem uma assinatura muito característica no APM: um pico repentino e sincronizado de latência/carga no banco de dados, correlacionado no tempo **exatamente** com a expiração de TTL de uma chave popular — não é gradual, é um degrau abrupto. No Redis, monitore a métrica `keyspace_misses` versus `keyspace_hits` : um pico repentino na taxa de miss (não um aumento gradual) é o sinal mais direto. Para o cenário de Redis indisponível, configure alertas separados: latência do Redis subindo (**INFO** `latencystats`) e, criticamente, a taxa de erro da sua API subindo **em rotas que usam cache** enquanto rotas que não usam cache continuam saudáveis — essa divergência é o que confirma que o Redis, e não o banco, é a causa raiz.

 **Desafio de Mentoria:** imagine que você implementou o lock distribuído com `SET NX` para resolver o cache stampede. Um Pleno júnior do seu time pergunta: "e se o processo que pegou o lock morrer (crash do pod, OOM kill) antes de liberar o lock ou popular o cache — o sistema fica travado para sempre até o `EX` expirar?" Como você responde, e que ajuste no design (TTL do lock, quem libera o lock, o que as requisições que esperam devem fazer se o timeout de espera estourar) você faria para que essa falha não vire uma indisponibilidade em cascata?

4.4 Design Patterns essenciais em Node.js

Observer — `EventEmitter` nativo

```
// node observer-eventemitter.js
const { EventEmitter } = require('events');

class PedidoEmitter extends EventEmitter {}
const pedidos = new PedidoEmitter();

// Múltiplos "observadores" reagindo ao mesmo evento, desacoplados entre si
pedidos.on('pedido:criado', (pedido) => {
  console.log(`[EMAIL] Enviando confirmação para pedido #${pedido.id}`);
});

pedidos.on('pedido:criado', (pedido) => {
  console.log(`[ESTOQUE] Reservando itens do pedido #${pedido.id}`);
});

pedidos.on('pedido:criado', (pedido) => {
  console.log(`[MÉTRICAS] Incrementando contador de pedidos`);
});

pedidos.emit('pedido:criado', { id: 101, total: 250 });
```

Retorno esperado:

```
[EMAIL] Enviando confirmação para pedido #101
[ESTOQUE] Reservando itens do pedido #101
[MÉTRICAS] Incrementando contador de pedidos
```

 **DICA DE PLENO:** o padrão Observer é a base de toda a arquitetura orientada a eventos do Node (`http.Server` , `stream` , etc). Usá-lo explicitamente no seu domínio (em vez de chamar as três funções em sequência dentro do controller) desacopla "o que aconteceu" de "quem reage a isso" — você pode adicionar um novo observador sem tocar no código que emite o evento.

🌟 Cenário de Falha em Produção

`EventEmitter.emit()` é **síncrono** por padrão — cada listener roda um após o outro, na mesma call stack, antes do `emit` retornar. Se um dos três listeners (`[EMAIL]` , `[ESTOQUE]` , `[MÉTRICAS]`) fizer uma chamada de rede bloqueante mal implementada, ou lançar uma exceção não tratada, isso **quebra os listeners seguintes** e propaga como exceção para quem chamou `emit` . Pior: se um listener é `async` mas o código que emite não sabe disso (`pedidos.emit(...)` não espera Promises), uma falha dentro do listener assíncrono vira uma `unhandled rejection` solta no ar, sem ninguém para capturá-la — o event emitter não tem noção nenhuma de que o handler retornou uma Promise.

🧠 Dica de Produção

Em produção, um `EventEmitter` sem listener de erro (`.on('error', ...)`) que recebe um `emit('error', ...)` **derruba o processo inteiro** — esse é um comportamento especial e intencional do Node para o evento `'error'` especificamente (não vale para outros nomes de evento). Sempre registre um listener de `'error'` em qualquer `EventEmitter` de longa duração. Para lógica de negócio crítica (como "reservar estoque"), não confie em Observer síncrono simples — prefira uma fila real (BullMQ, SQS) para os efeitos colaterais que precisam de garantia de entrega, e reserve `EventEmitter` para efeitos "best-effort" dentro do mesmo processo, como métricas e logs.

Decorator — função wrapper pura (sem TS decorators experimentais)

```
// npx ts-node decorator-function.ts

// Decorator funcional: recebe uma função e retorna outra função "decorada"
function comLog<T extends (...args: any[]) => any>(fn: T): T {
  return function (...args: Parameters<T>): ReturnType<T> {
    console.log(`[LOG] Chamando "${fn.name}" com args:`, args);
    const resultado = fn(...args);
    console.log(`[LOG] "${fn.name}" retornou:`, resultado);
    return resultado;
  } as T;
}

function somar(a: number, b: number): number {
  return a + b;
}

const somarComLog = comLog(somar);
somarComLog(2, 3);
```

Retorno esperado:

```
[LOG] Chamando "somar" com args: [ 2, 3 ]
[LOG] "somar" retornou: 5
```

💡 **DICA DE PLENO:** essa abordagem (decorator como função pura) é mais portátil e previsível que os decorators experimentais do TypeScript (`@Decorator` , ainda dependentes de flag experimental até recentemente) — funciona em qualquer runtime JS sem configuração especial, e é o mesmo princípio por trás de middlewares do Express e HOCs (Higher-Order Components) do React.

💥 Cenário de Falha em Produção

O exemplo `comLog` não trata funções `async` corretamente: se `fn` retornar uma Promise, `console.log('retornou:', resultado)` loga o **objeto Promise pendente**, não o valor resolvido — um bug sutil que passa despercebido em teste manual (porque o desenvolvedor vê "alguma coisa" logada) mas gera logs inúteis em produção. Um decorator de `retry` com esse mesmo problema é mais grave: se ele não propagar corretamente erros assíncronos, pode "engolir" uma falha real e reportar sucesso, mascarando um problema que deveria ter disparado alerta.

🧠 Dica de Produção

Decorators/wrappers de logging e retry ficam no **caminho crítico** de toda chamada que envolvem — um bug neles não afeta uma feature isolada, afeta **tudo** que passa por ali. Antes de aplicar um decorator genérico em produção, escreva testes específicos para o caso assíncrono (`fn` retornando Promise resolvida e rejeitada) — é o ponto cego mais comum nesse padrão. Em logs estruturados gerados por um decorator de logging, sempre inclua um identificador de correlação (`requestId` / `traceId`) como argumento implícito, não apenas os argumentos da função — do contrário, o log gerado pelo decorator fica "solto", sem como ser correlacionado com o resto da requisição no seu APM.

Singleton — module caching do Node

```
// conexao-banco.js
console.log('Inicializando conexão com o banco (isso só deve rodar UMA vez)...');
module.exports = { conexaoId: Math.random() };
```

```
// node singleton-module-cache.js
const conexaoA = require('./conexao-banco');
const conexaoB = require('./conexao-banco');

console.log('conexaoA === conexaoB?', conexaoA === conexaoB);
console.log('Mesmo conexaoId?', conexaoA.conexaoId === conexaoB.conexaoId);
```

Retorno esperado:

```
Inicializando conexão com o banco (isso só deve rodar UMA vez)...
conexaoA === conexaoB? true
Mesmo conexaoId? true
```

 **DICA DE PLENO:** o Node.js **já cacheia módulos** por padrão (via **require** / resolução de módulos ES) — a primeira vez que um módulo é importado, ele é executado e o resultado fica em cache; importações seguintes reutilizam a mesma instância. Isso significa que, na prática, você **raramente precisa** implementar manualmente o padrão Singleton clássico (com **getInstance()**) em Node — basta exportar a instância já criada no módulo.

Por baixo dos panos

O cache de módulos do Node é indexado pelo **caminho resolvido absoluto** do arquivo (**require.cache**, uma chave por path absoluto). Isso tem uma implicação sutil e traiçoeira: se o mesmo pacote existir em duas versões diferentes na árvore de **node_modules** (comum com dependências transitivas conflitantes, ou em monorepos com múltiplos **package.json**), você pode acabar com **duas instâncias "singleton"** diferentes do que deveria ser um único objeto — cada **require** resolvendo para um caminho de arquivo físico diferente, mesmo que o nome do pacote seja idêntico.

🌟 Cenário de Falha em Produção

Uma conexão de banco "singleton" (como no exemplo `conexao-banco.js`) parece segura, mas se seu processo Node roda com **cluster mode** (via `cluster` nativo ou PM2 em modo cluster para aproveitar múltiplos cores), cada processo *worker* do cluster é um **processo do SO inteiramente separado**, com sua própria instância do V8 e, portanto, seu próprio módulo "singleton" — você não tem uma conexão de banco, você tem N conexões (uma por worker), cada uma pensando ser única. Isso costuma surpreender times que escalam horizontalmente pela primeira vez: "por que temos mais conexões abertas no Postgres do que o singleton deveria permitir?" — a resposta é que singleton em Node é singleton **por processo**, nunca por máquina ou por cluster.

🧠 Dica de Produção

Ao investigar "número de conexões inesperado" no banco (visível em `pg_stat_activity` no Postgres, por exemplo), sempre correlacione com o número de processos/workers da sua aplicação (`pm2 list`, ou réplicas do pod no Kubernetes) antes de suspeitar de vazamento de conexão no código — multiplique o "esperado por processo" pelo número de processos reais rodando. Esse é um dos erros de diagnóstico mais comuns de Plenos migrando de aplicações single-process para arquiteturas com múltiplas réplicas.

🤔 **Desafio de Mentoria:** você tem quatro padrões nesta seção — Repository, wrapper de erro, Cache-Aside, e os três design patterns (Observer/Decorator/Singleton). Se sua startup está migrando de 1 instância da API para 10 réplicas atrás de um load balancer (para suportar os 50k usuários), qual desses padrões, do jeito que foi implementado nos exemplos acima, você precisaria **reavaliar primeiro** antes de escalar horizontalmente — e por quê? Pense em qual deles carrega uma suposição implícita de "existe só um processo rodando".

🔗 Referências

- [Prisma Docs — Repository-like patterns](#)
- [Express — Error Handling](#)
- [Redis Docs — Caching patterns](#)
- [Node.js EventEmitter \(docs\)](#)
- [Refactoring Guru — Design Patterns](#)

5. Testes

5.1 Teste unitário com Vitest mockando um repositório

```
// npm install -D vitest
// npx vitest run usuario-service.test.ts

import { describe, it, expect, vi } from 'vitest';

interface UsuarioRepository {
  buscarPorId(id: number): Promise<{ id: number; nome: string } | null>;
}

class UsuarioService {
  constructor(private repo: UsuarioRepository) {}

  async obterNomeFormatado(id: number): Promise<string> {
    const usuario = await this.repo.buscarPorId(id);
    if (!usuario) throw new Error('Usuário não encontrado');
    return usuario.nome.toUpperCase();
  }
}

describe('UsuarioService', () => {
  it('retorna o nome formatado quando o usuário existe', async () => {
    const repoMock: UsuarioRepository = {
      buscarPorId: vi.fn().mockResolvedValue({ id: 1, nome: 'ana' }),
    };
    const service = new UsuarioService(repoMock);
    const resultado = await service.obterNomeFormatado(1);

    expect(resultado).toBe('ANA');
    expect(repoMock.buscarPorId).toHaveBeenCalledTimes(1);
    expect(repoMock.buscarPorId).toHaveBeenCalledWith(1);
  });

  it('lança erro quando o usuário não existe', async () => {
    const repoMock: UsuarioRepository = {
      buscarPorId: vi.fn().mockResolvedValue(null),
    };
    const service = new UsuarioService(repoMock);
    await expect(service.obterNomeFormatado(99)).rejects.toThrow('Usuário não encontrado');
  });
});
```

Retorno esperado:

- ✓ UsuarioService > retorna o nome formatado quando o usuário existe
- ✓ UsuarioService > lança erro quando o usuário não existe

Test Files 1 passed (1)
Tests 2 passed (2)

5.2 Teste de integração com Supertest

```
// npm install -D supertest @types/supertest
// npm install express
// npx vitest run app.integration.test.ts

import request from 'supertest';
import express from 'express';
import { describe, it, expect } from 'vitest';

function criarApp() {
  const app = express();
  app.use(express.json());

  app.post('/usuarios', (req, res) => {
    const { nome, email } = req.body;
    if (!nome || !email) {
      return res.status(400).json({ erro: 'nome e email são obrigatórios' });
    }
    return res.status(201).json({ id: 1, nome, email });
  });

  return app;
}

describe('POST /usuarios (integração)', () => {
  const app = criarApp();

  it('cria um usuário com dados válidos', async () => {
    const resposta = await request(app).post('/usuarios').send({ nome: 'Ana', email: 'ana@email.com' });
    expect(resposta.status).toBe(201);
    expect(resposta.body).toEqual({ id: 1, nome: 'Ana', email: 'ana@email.com' });
  });

  it('retorna 400 quando faltam dados', async () => {
    const resposta = await request(app).post('/usuarios').send({ nome: 'Ana' });
    expect(resposta.status).toBe(400);
    expect(resposta.body.erro).toContain('obrigatórios');
  });
});
```

Retorno esperado:

- ✓ `POST /usuarios (integração)` > cria um usuário com dados válidos
- ✓ `POST /usuarios (integração)` > retorna 400 quando faltam dados

Test Files 1 passed (1)
Tests 2 passed (2)

 **DICA DE PLENO:** teste unitário mocka dependências e testa lógica isolada (rápido, roda aos milhares). Teste de integração sobe a aplicação real e testa o comportamento observável de fora — mais lento, mas pega bugs de "juntar as peças".

Por baixo dos panos

`vi.fn().mockResolvedValue(...)` cria um mock que **sempre** resolve com sucesso, imediatamente, sem passar pelas fases reais do event loop que uma chamada de rede/banco passaria (sem I/O real, sem espera na fase `poll`). Isso é ótimo para velocidade, mas significa que testes unitários **não pegam** bugs de concorrência real — race conditions, timeouts, ou comportamento sob latência variável simplesmente não existem no mundo do mock, porque tudo resolve de forma síncrona e determinística demais.

Cenário de Falha em Produção

Um Pleno escreve um teste unitário robusto para `UsuarioService`, mockando o repositório com `mockResolvedValue`. A suíte passa 100%, o PR é aprovado. Em produção, sob carga real, o banco real leva 200ms para responder (não 0ms como o mock), e **duas** requisições concorrentes chamando `obterNomeFormatado` para o mesmo usuário causam uma condição de corrida em outro trecho do código que usa esse resultado para atualizar um cache — algo que o teste unitário jamais revelaria, porque o mock nunca introduziu latência real o suficiente para expor a janela de corrida.

Dica de Produção

Testes unitários com mocks bem-feitos garantem **lógica correta**, não **comportamento correto sob concorrência real**. Para isso, testes de integração (seção 5.2, subindo a aplicação real com banco/Redis via `testcontainers` ou similar) e, principalmente, testes de carga em ambiente de staging (seção 10.2, `autocannon`) são complementares — nenhum substitui o outro. Se um incidente em produção só reproduz sob carga concorrente real e nunca em teste local com uma requisição por vez, é sinal de que a suíte de testes está parada só no nível unitário e precisa de cobertura de concorrência/carga.

5.3 Property-Based Testing com **fast-check**

Testes tradicionais checam **exemplos específicos** ("para o input X, espero o output Y"). Property-based testing gera **centenas de inputs aleatórios** e checa se uma **propriedade geral** se mantém verdadeira para todos eles — ótimo para achar edge cases que você nem pensaria em testar manualmente.

```
// npm install -D fast-check vitest
// npx vitest run property-based.test.ts

import { describe, it, expect } from 'vitest';
import fc from 'fast-check';

function ehPositivo(n: number): boolean {
  return n > 0;
}

describe('ehPositivo (property-based)', () => {
  it('deve retornar true para QUALQUER número maior que zero', () => {
    fc.assert(
      fc.property(
        fc.float({ min: Math.fround(0.0001), max: 1_000_000, noNaN: true }), // gera centenas de f
        (numeroAleatorio) => {
          expect(ehPositivo(numeroAleatorio)).toBe(true);
        }
      )
    );
  });

  it('deve retornar false para QUALQUER número menor ou igual a zero', () => {
    fc.assert(
      fc.property(
        fc.float({ min: -1_000_000, max: 0, noNaN: true }),
        (numeroAleatorio) => {
          expect(ehPositivo(numeroAleatorio)).toBe(false);
        }
      )
    );
  });
});
```

Retorno esperado:

- ✓ ehPositivo (property-based) > deve retornar true para QUALQUER número maior que zero
- ✓ ehPositivo (property-based) > deve retornar false para QUALQUER número menor ou igual a zero

```
Test Files  1 passed (1)
Tests      2 passed (2)
```

 **DICA DE PLENO:** se `ehPositivo` tivesse um bug (por exemplo, tratando `0` incorretamente, ou tendo problema de arredondamento com floats muito pequenos), o `fast-check` **encontraria automaticamente** o menor caso que quebra a propriedade (processo chamado *shrinking*) e mostraria exatamente esse valor no relatório de falha — algo que testes com exemplos fixos dificilmente cobririam sem uma lista gigante e manual de casos.

Cenário de Falha em Produção

Property-based testing é especialmente valioso em código de cálculo financeiro (juros, descontos, arredondamento de valores monetários), justamente onde bugs de ponto flutuante (`0.1 + 0.2 !== 0.3` em JavaScript) causam divergências de centavos que se acumulam em milhões de transações. Um Pleno testa `calcularJuros` com 3 exemplos fixos, todos passam, o código vai para produção — mas existe um valor específico de entrada (que só o `fast-check`, gerando centenas de floats aleatórios, encontraria) onde o arredondamento produz um resultado 1 centavo diferente do esperado. Isoladamente é irrelevante; multiplicado por milhões de transações mensais, vira uma divergência contábil real que o time financeiro vai gastar dias investigando.

Dica de Produção

Property-based tests não geram um "ambiente de produção" para monitorar — o valor deles é **prevenir** a classe inteira de bugs de edge case antes do deploy, não diagnosticar depois. Mas quando um bug desse tipo *já* está em produção (porque a suíte não tinha PBT), a pista nos logs costuma ser divergências pequenas e inconsistentes em valores calculados — não erros, não exceções, apenas "os números não batem" em relatórios de reconciliação. Se seu time investiga esse tipo de discrepância com frequência, é um forte sinal de que a função de cálculo em questão merece um teste de propriedade retroativo, cobrindo a faixa real de valores observados em produção.

 **Desafio de Mentoria:** dado que testes unitários com mock rodam em milissegundos mas não pegam problemas de concorrência real, e testes de integração pegam mais bugs mas são mais lentos — em um pipeline de CI que precisa rodar em menos de 5 minutos para não atrapalhar o fluxo do time, como você decidiria a proporção entre testes unitários, de integração e property-based? O que você move para rodar apenas em um pipeline noturno/separado, e o que precisa obrigatoriamente bloquear o merge?

Referências

- [Vitest — Mocking](#)
 - [Supertest \(GitHub\)](#)
 - [fast-check — Property-Based Testing](#)
 - [Martin Fowler — Unit Test vs Integration Test](#)
-

6. Segurança Essencial

6.1 Hash de senha com bcrypt

```
// npm install bcrypt
// npm install -D @types/bcrypt
// npx ts-node bcrypt-exemplo.ts

import bcrypt from 'bcrypt';

const SALT_ROUNDS = 12;

async function main() {
  const senhaPura = 'MinhaSenh@123';
  const hash = await bcrypt.hash(senhaPura, SALT_ROUNDS);
  console.log('Hash gerado:', hash);

  const senhaCorreta = await bcrypt.compare('MinhaSenh@123', hash);
  const senhaErrada = await bcrypt.compare('senhaErrada', hash);

  console.log('Senha correta confere?', senhaCorreta);
  console.log('Senha errada confere?', senhaErrada);
}

main();
```

Retorno esperado (o hash muda a cada execução, pois o salt é aleatório):

```
Hash gerado: $2b$12$KIXQ7z3n9y8s1f...V8gk4uT4uWq60
Senha correta confere? true
Senha errada confere? false
```

⚠️ **ARMADILHA COMUM:** usar **MD5** ou **SHA-256** puro para senhas é grave — são **rápidos demais**, vulneráveis a força bruta com GPUs. **bcrypt** (assim como **argon2** / **scrypt**) é propositalmente lento e usa salt automático, tornando ataques de dicionário/rainbow table inviáveis.



Por baixo dos panos

`bcrypt.hash` com `SALT_ROUNDS = 12` não é "12 vezes mais lento" — é **exponencial**: o custo dobra a cada round adicional (2^{12} iterações do algoritmo Blowfish internamente). Isso é uma decisão de trade-off consciente entre segurança e latência: `bcrypt` roda em uma operação **síncrona e CPU-bound** por natureza do algoritmo — a versão `async` do pacote `bcrypt` (não `bcryptjs`, que é puro JS) delega o trabalho pesado para a thread pool da libuv (não para o event loop principal), especificamente o pool de threads usado também para operações de arquivo — por padrão, apenas **4 threads** (`UV_THREADPOOL_SIZE`).

🌟 Cenário de Falha em Produção

Em uma rota de login com alto tráfego (50k usuários ativos, picos de login pela manhã), cada `bcrypt.hash` / `bcrypt.compare` ocupa uma das 4 threads do pool da libuv por dezenas de milissegundos. Se você também usa `fs.readFile` assíncrono ou DNS lookup em outras partes do sistema (ambos também competem pelo mesmo pool de 4 threads por padrão), um pico de logins simultâneos pode **esgotar o thread pool inteiro**, fazendo operações completamente não relacionadas (leitura de arquivo de configuração, resolução de DNS de uma chamada a serviço externo) enfileirarem atrás dos hashes de senha — um efeito de "tudo lento ao mesmo tempo" difícil de conectar à causa raiz (login) se você não souber que ambos compartilham o mesmo pool.

🧠 Dica de Produção

Se sua API tem picos de latência correlacionados com picos de login, meça o tamanho da fila do thread pool da libuv (não há métrica nativa direta, mas você pode inferir via `perf_hooks` medindo o delay de operações que passam pelo pool, como `fs.stat`) e considere aumentar `UV_THREADPOOL_SIZE` (variável de ambiente, até um limite razoável como 16-32) se o processo tiver CPU sobrando. Em qualquer APM, se você vir `bcrypt.compare` / `bcrypt.hash` aparecendo como span lento (dezenas a centenas de ms) e correlacionado com lentidão *em outras rotas não relacionadas a autenticação*, essa é a assinatura de contenção no thread pool compartilhado — não conclua "bcrypt está lento" isoladamente, investigue o que mais compete pelo mesmo pool.

6.2 JWT: geração, verificação e refresh token

```
// npm install jsonwebtoken
// npm install -D @types/jsonwebtoken
// npx ts-node jwt-exemplo.ts

import jwt from 'jsonwebtoken';

const SEGredo_ACCESS = 'chave-secreta-access-token';
const SEGredo_REFRESH = 'chave-secreta-refresh-token';

function gerarTokens(usuarioId: number) {
  const accessToken = jwt.sign({ sub: usuarioId }, SEGredo_ACCESS, { expiresIn: '15m' });
  const refreshToken = jwt.sign({ sub: usuarioId }, SEGredo_REFRESH, { expiresIn: '7d' });
  return { accessToken, refreshToken };
}

function verificarAccessToken(token: string) {
  return jwt.verify(token, SEGredo_ACCESS) as { sub: number };
}

function renovarAccessToken(refreshToken: string) {
  const payload = jwt.verify(refreshToken, SEGredo_REFRESH) as { sub: number };
  return jwt.sign({ sub: payload.sub }, SEGredo_ACCESS, { expiresIn: '15m' });
}

const { accessToken, refreshToken } = gerarTokens(42);
console.log('Access Token:', accessToken.slice(0, 30) + '...');

const dadosDecodificados = verificarAccessToken(accessToken);
console.log('Payload decodificado:', dadosDecodificados);

const novoAccessToken = renovarAccessToken(refreshToken);
console.log('Novo Access Token gerado via refresh:', novoAccessToken.slice(0, 30) + '...');

try {
  jwt.verify('token.invalido.aqui', SEGredo_ACCESS);
} catch (erro: any) {
  console.log('Erro esperado ao verificar token inválido:', erro.message);
}
```

Retorno esperado:

```
Access Token: eyJhbGciOiJIUzI1NiIsInR5cCI6I..
Payload decodificado: { sub: 42, iat: 1234567890, exp: 1234568790 }
Novo Access Token gerado via refresh: eyJhbGciOiJIUzI1NiIsInR5cCI6I..
Erro esperado ao verificar token inválido: jwt malformed
```

Fluxo conceitual:

1. Login com senha → servidor emite `accessToken` (curta duração) + `refreshToken` (longa duração, idealmente em cookie `httpOnly`).
2. Cliente usa `accessToken` no header `Authorization` em cada requisição.
3. Quando expira (401), o cliente chama `/refresh` enviando o `refreshToken`.
4. Servidor valida o `refreshToken` e emite um `accessToken` novo — sem pedir senha de novo.
5. Se o `refreshToken` também expirar/for revogado, o usuário precisa logar de novo.

💡 **DICA DE PLENO:** ter **dois segredos diferentes** limita o dano: se o segredo de access token vazar, um atacante só forja tokens de curta duração — não consegue gerar refresh tokens válidos de 7 dias.

Por baixo dos panos

Um JWT não é criptografado — é **assinado**. O payload (`{ sub: 42, iat, exp }`) está em Base64URL, legível por qualquer pessoa que interceptar o token (abra qualquer JWT em jwt.io e veja). A assinatura (HMAC-SHA256 no exemplo, usando o segredo) só garante **integridade** (ninguém alterou o payload sem o segredo), não **confidencialidade**. É um erro de Pleno recorrente colocar dados sensíveis (senha, dados pessoais completos) dentro do payload do JWT "porque está assinado, então é seguro" — qualquer um com o token consegue ler o conteúdo, só não consegue *forjar* um novo válido.

Cenário de Falha em Produção

O exemplo do material usa `jwt.verify` sem revogação — uma vez emitido, um `accessToken` é válido até expirar, **mesmo que o usuário seja banido, mude a senha, ou o token vaze**, porque JWT é *stateless* por design (essa é a vantagem de performance dele: nenhuma consulta a banco para validar). Se sua startup precisa de "logout imediato" ou "revogar acesso de um usuário comprometido agora", um JWT puro não resolve — o token forjado antes da revogação continua válido pelos próximos 15 minutos (ou o TTL configurado), permitindo ações não autorizadas nesse intervalo.

 Dica de Produção

Para mitigar isso sem perder a vantagem de performance do JWT, mantenha uma **denylist** leve no Redis (`SET revogado:<jti> true EX <tempo_restante_do_token>`) para os poucos casos que exigem revogação imediata (logout forçado, banimento) — verificada apenas nesses fluxos sensíveis, não em toda requisição. Em produção, monitore o padrão de erros `jwt expired` versus `jwt malformed / invalid signature` separadamente nos logs: um pico de `invalid signature` pode indicar tentativa de forjar tokens (ataque ativo) e merece alerta de segurança; `jwt expired` em alto volume é normal e só indica clientes não renovando a tempo.

6.3 SQL Injection: vulnerável vs parametrizada

❌ Jeito Júnior vs ✅ Jeito Pleno

```
// npm install pg
// npx ts-node sql-injection.ts

import { Pool } from 'pg';
const pool = new Pool();

// ❌ JEITO JÚNIOR – concatenação direta de string. VULNERÁVEL.
async function buscarUsuarioVulneravel(emailDigitado: string) {
  const query = `SELECT * FROM usuarios WHERE email = '${emailDigitado}'`;
  console.log('Query executada:', query);
  return pool.query(query);
}

// ✅ JEITO PLENO – query parametrizada
async function buscarUsuarioSeguro(emailDigitado: string) {
  const query = 'SELECT * FROM usuarios WHERE email = $1';
  console.log('Query executada:', query, '| Parâmetro:', emailDigitado);
  return pool.query(query, [emailDigitado]);
}

const entradaMaliciosa = "' OR '1'='1";
console.log('--- Vulnerável ---');
console.log(`SELECT * FROM usuarios WHERE email = '${entradaMaliciosa}'`);
console.log('--- Segura (parametrizada) ---');
console.log('SELECT * FROM usuarios WHERE email = $1 → parâmetro tratado como valor literal');
```

Retorno esperado:

```

--- Vulnerável ---
SELECT * FROM usuarios WHERE email = '' OR '1'='1'
--- Segura (parametrizada) ---
SELECT * FROM usuarios WHERE email = $1 → parâmetro tratado como valor literal

```

Por que a diferença: na versão vulnerável, o input do usuário vira **parte do código SQL**. Na parametrizada, o driver separa **estrutura da query** de **dado**, enviados separadamente ao servidor de banco — impossível injetar SQL arbitrário.

⚠ **ARMADILHA COMUM:** ORMs como Prisma e TypeORM protegem contra SQL Injection **automaticamente** nos métodos padrão — mas ambos oferecem "escape hatches" para SQL bruto (`$queryRawUnsafe`, query bruta). Se você concatenar strings dentro desses métodos "unsafe", a proteção deixa de existir.

🔥 Cenário de Falha em Produção

Um caso real e comum: um Pleno usa Prisma para 95% do sistema, mas para um relatório complexo (agregação que o Prisma Client não modela bem) recorre a

`$queryRawUnsafe(\ SELECT * FROM pedidos WHERE status = '${statusFiltro}')`, interpolando um filtro que vem de um query param da URL. O código passa em todos os testes funcionais (ninguém testa injeção de SQL manualmente todo PR), o PR é aprovado porque "o resto do arquivo usa Prisma direito", e a vulnerabilidade só é encontrada meses depois — por um pentest, ou pior, por um atacante real, que consegue extrair a tabela inteira de usuários com um payload tipo `' UNION SELECT senha_hash, email, null FROM usuarios --``.

 Dica de Produção

SQL Injection bem-sucedido em produção geralmente aparece nos logs do banco como queries **anormalmente longas ou com sintaxe incomum** (**UNION** , comentários **--** , múltiplos **;**) vindas do usuário da aplicação — se você tem **pg_stat_statements** ou logging de queries lentas/incomuns habilitado, isso é auditável retroativamente.

Preventivamente, ferramentas de SAST (Semgrep, Snyk Code) configuradas no CI para bloquear qualquer uso de **\$queryRawUnsafe** /concatenação de string em métodos de query bruta são a defesa mais barata — trate qualquer alerta desse tipo como bloqueante de merge, nunca como "vou revisar depois".

 **Desafio de Mentoria:** você é o revisor de um PR que adiciona autenticação a uma nova API interna (consumida só por outros serviços da própria empresa, nunca por usuários finais diretamente). O autor do PR argumenta que, por ser "tráfego interno confiável", pode pular o hashing de senha com bcrypt e usar um token estático simples em variável de ambiente, para "simplificar e ganhar performance". Que perguntas você faz antes de aprovar — pensando em como uma startup de alto crescimento tipicamente evolui (contratação de novos serviços, terceirização, possíveis credenciais vazadas em repositórios) — para decidir se esse trade-off é aceitável ou se é dívida técnica de segurança disfarçada de otimização?

 Referências

- [OWASP — SQL Injection Prevention Cheat Sheet](#)
- [bcrypt \(npm\)](#)
- [jwt.io — Introduction to JSON Web Tokens](#)
- [node-postgres — Parameterized Query](#)

7. Docker, Git e Linux

7.1 Docker: Multi-stage build para app Node.js

O objetivo do multi-stage build é ter uma imagem final **enxuta**, sem dependências de desenvolvimento, ferramentas de build, ou código-fonte TypeScript não compilado.

```
# Dockerfile
# --- ESTÁGIO 1: build ---
# Usa uma imagem completa, com tudo necessário para compilar o TypeScript
FROM node:20-alpine AS build

WORKDIR /app

# Copia só os manifestos primeiro – aproveita cache do Docker se as dependências não mudarem
COPY package.json package-lock.json ./
RUN npm ci

# Agora copia o resto do código-fonte
COPY . .
RUN npm run build # gera a pasta dist/ com o JS compilado

# --- ESTÁGIO 2: produção ---
# Imagem final, enxuta – SEM devDependencies, SEM código TS fonte
FROM node:20-alpine AS production

WORKDIR /app
ENV NODE_ENV=production

# Copia só o necessário do estágio anterior
COPY package.json package-lock.json ./
RUN npm ci --omit=dev

COPY --from=build /app/dist ./dist

# Healthcheck: o orquestrador (Docker/K8s) usa isso para saber se o container está saudável
HEALTHCHECK --interval=30s --timeout=5s --start-period=10s \
  CMD node -e "require('http').get('http://localhost:3000/health', (r) => process.exit(r.statusCode))"

# Rodar como usuário não-root – boa prática de segurança
USER node

EXPOSE 3000
CMD ["node", "dist/main.js"]
```

Retorno esperado ao rodar **docker build -t minha-api .** e depois **docker run -p 3000:3000 minha-api :**

```
[+] Building 12.4s (14/14) FINISHED
=> [build 1/5] FROM docker.io/library/node:20-alpine
=> [build 4/5] RUN npm ci
=> [build 5/5] RUN npm run build
=> [production 3/4] RUN npm ci --omit=dev
=> exporting to image
=> => naming to docker.io/library/minha-api
```

Servidor rodando na porta 3000

⚠ **ARMADILHA COMUM:** esquecer o `.dockerignore` (excluindo `node_modules`, `.git`, `dist` locais) faz o build copiar arquivos desnecessários para dentro do contexto, deixando a imagem gigante e o build lento. Sempre crie um `.dockerignore` espelhando o `.gitignore`.

Por baixo dos panos

Cada instrução do Dockerfile cria uma **camada (layer)** imutável, e o Docker cacheia camadas por hash do conteúdo + instrução anterior. É por isso que `COPY package.json package-lock.json ./` vem **antes** de `COPY . .`: se só o código-fonte mudar (não as dependências), o Docker reaproveita a camada do `npm ci` inteira, pulando a reinstalação. Inverter essa ordem (copiar tudo antes de instalar) invalida o cache a cada mudança de código, mesmo trivial, forçando reinstalação completa de `node_modules` em todo build — um erro comum que transforma builds de 10 segundos em 3 minutos.

💥 Cenário de Falha em Produção

`HEALTHCHECK` com `start-period=10s` assume que a aplicação sobe em 10 segundos. Se sua API faz uma migração de banco automática no boot (`prisma migrate deploy` antes de `node dist/main.js`) e o volume de dados cresce ao ponto da migração levar 40 segundos, o orquestrador (Docker/Kubernetes) começa a marcar o container como "unhealthy" **antes dele terminar de subir**, e — dependendo da política de restart configurada — pode reiniciar o container repetidamente, num loop de crash que nunca deixa a aplicação terminar de inicializar, porque cada tentativa é interrompida antes de completar a migração.



Dica de Produção

Em produção, um container preso em `CrashLoopBackOff` (termo do Kubernetes) com healthcheck falhando é diferenciável de "aplicação com bug" observando os **logs do container antes do kill**: se o log sempre para no mesmo ponto ("Rodando migração...") sem nunca chegar em "Servidor rodando", é timeout de `start-period`, não bug de lógica. A correção é ajustar `start-period` para acomodar o pior caso realista de boot (não o caso médio), ou — melhor — separar migração de banco do boot da aplicação (rodar como um `Job` /step de deploy anterior, não dentro do `CMD` do container da API).

docker-compose.yml : app + Redis + Postgres

```
# docker-compose.yml
version: '3.9'

services:
  api:
    build: .
    ports:
      - '3000:3000'
    environment:
      DATABASE_URL: postgres://usuario:senha@postgres:5432/meubanco
      REDIS_URL: redis://redis:6379
    depends_on:
      postgres:
        condition: service_healthy
      redis:
        condition: service_healthy
    networks:
      - app-network

  postgres:
    image: postgres:16-alpine
    environment:
      POSTGRES_USER: usuario
      POSTGRES_PASSWORD: senha
      POSTGRES_DB: meubanco
    volumes:
      - postgres-data:/var/lib/postgresql/data # volume nomeado: persiste dados entre restarts
    healthcheck:
      test: ['CMD-SHELL', 'pg_isready -U usuario']
      interval: 5s
      timeout: 5s
      retries: 5
    networks:
      - app-network

  redis:
    image: redis:7-alpine
    healthcheck:
      test: ['CMD', 'redis-cli', 'ping']
      interval: 5s
      timeout: 3s
      retries: 5
    networks:
      - app-network

volumes:
  postgres-data: # sem isso, os dados do Postgres somem a cada `docker compose down`

networks:
  app-network: # rede isolada — os serviços se enxergam pelo NOME (postgres, redis), não por IP
    driver: bridge
```

Retorno esperado ao rodar `docker compose up`:

```
✓ Network app_app-network Created
✓ Volume "app_postgres-data" Created
✓ Container app-postgres-1 Healthy
✓ Container app-redis-1 Healthy
✓ Container app-api-1 Started
api-1 | Servidor rodando na porta 3000
```

💡 **DICA DE PLENO:** o `depends_on` com `condition: service_healthy` é o que diferencia Pleno de Júnior aqui — sem isso, o `depends_on` simples só garante que o `container` subiu, não que o **serviço dentro dele já está pronto para aceitar conexões**. Sua API pode tentar conectar no Postgres antes dele terminar de inicializar e crashar no boot.

🔥 Cenário de Falha em Produção

`docker-compose.yml` é uma ferramenta de **desenvolvimento local**, não de produção — mas times de startups em crescimento rápido frequentemente "promovem" a mesma configuração para produção sem revisar suposições que fazem sentido só localmente. Um exemplo real: `healthcheck` com `retries: 5` e `interval: 5s` (25 segundos de tolerância) é razoável para sua máquina local, mas em produção, sob picos de carga real, um Postgres saudável mas momentaneamente sobrecarregado pode não responder ao `pg_isready` dentro dessa janela — o orquestrador reinicia um banco de dados **saudável**, apenas ocupado, criando uma instabilidade auto-infligida bem no momento de maior tráfego.

🧠 Dica de Produção

Ao migrar de `docker-compose` local para um orquestrador de produção real (Kubernetes, ECS), revise **todos** os parâmetros de `healthcheck` com dados reais de produção (p99 de tempo de resposta sob carga), não com os valores "que funcionaram no meu laptop". No Kubernetes, a distinção entre `livenessProbe` (reinicia o pod se falhar) e `readinessProbe` (só tira do balanceamento, sem reiniciar) é crucial aqui — um serviço sobrecarregado deveria sair do balanceamento (`readiness` falhando) para aliviar a carga, não ser reiniciado (`liveness` falhando), que só piora a situação ao perder o estado/conexões que já existiam.

7.2 Git: Rebase interativo, Cherry-pick e Conflitos

Rebase interativo (squash, reword) vs Merge

```
# Cenário: você fez 4 commits "sujos" numa branch de feature
# e quer limpar o histórico antes de abrir o PR.

git log --oneline
# a1b2c3d fix typo
# e4f5g6h wip: ajustando validação
# h7i8j9k wip: mais um ajuste
# k1l2m3n feat: implementa validação de CPF

git rebase -i HEAD~4
```

O editor abre com algo assim (você edita as ações):

```
pick k1l2m3n feat: implementa validação de CPF
squash h7i8j9k wip: mais um ajuste
squash e4f5g6h wip: ajustando validação
reword a1b2c3d fix typo
```

Retorno esperado após salvar e resolver o reword:

```
[detached HEAD 9f8e7d6] feat: implementa validação de CPF
Date: ...
3 files changed, 42 insertions(+), 5 deletions(-)
Successfully rebased and updated refs/heads/feature/validacao-cpf.
```

O histórico vira um único commit limpo (`feat: implementa validação de CPF`), em vez de 4 commits de "wip".

Rebase vs Merge — quando usar cada um:

	<code>git rebase</code>	<code>git merge</code>
Histórico	Linear, reescreve commits (novos hashes)	Preserva o histórico real, cria commit de merge
Uso típico	Limpar sua branch de feature antes de abrir PR, ou atualizar sua branch com a <code>main</code>	Integrar uma branch de feature finalizada na <code>main</code>
Regra de ouro	Nunca dar rebase em commits que já foram <i>pushed</i> e compartilhados com outras pessoas	Seguro em qualquer cenário, inclusive branches compartilhadas

⚠ **ARMADILHA COMUM:** dar `git rebase` numa branch que outras pessoas já puxaram (`git pull`) reescreve o histórico e causa conflitos generalizados para todo mundo que já tinha os commits antigos. Regra prática: rebase é para "arrumar sua casa antes de mostrar pra visita" (sua branch local, não compartilhada); merge é para "juntar duas casas" (branches já públicas).

Por baixo dos panos

Cada commit no Git é identificado por um hash SHA-1/SHA-256 calculado a partir do **conteúdo do commit** (diff, mensagem, autor, timestamp, e o hash do commit pai).

Rebase não "move" commits — ele **recria** cada commit do zero, com um novo pai, o que necessariamente gera um novo hash, mesmo que o conteúdo do código seja idêntico. É por isso que rebase é "reescrita de histórico": os commits antigos ainda existem no **reflog** local por um tempo, mas deixam de fazer parte do histórico "oficial" da branch — qualquer referência externa a esses hashes antigos (um link de PR, um `git log` salvo, uma branch de outra pessoa baseada nesses commits) quebra.

💥 Cenário de Falha em Produção

Um cenário real em times distribuídos: um desenvolvedor dá `git push --force` numa branch de feature compartilhada após um rebase, sem avisar o time. Um colega que já tinha puxado a versão antiga da branch faz `git pull` (sem `--rebase`), o Git tenta fazer merge das duas históricas divergentes (a antiga local e a nova remota, que tecnicamente não têm ancestral comum reconhecível por hash), e o resultado é um emaranhado de conflitos gigantesco — ou pior, sem perceber o problema, o colega faz push de volta, **reintroduzindo os commits antigos** que o rebase original tentou eliminar, duplicando o histórico.

Dica de Produção

Antes de qualquer `git push --force` numa branch compartilhada, use `git push --force-with-lease` — ele falha (protegendo você) se alguém mais fez push depois da última vez que você buscou a branch, em vez de sobrescrever cegamente o trabalho alheio. Como regra de time, force-push só deveria ser permitido em branches de feature pessoais, nunca em `main` / `develop` / branches de release — a maioria das plataformas Git (GitHub, GitLab) permite bloquear force-push nessas branches via regra de proteção, o que é mais confiável que confiar na disciplina de todo o time.

Cherry-pick

```
# Cenário: um commit de hotfix crítico foi feito na branch `main`,
# e você precisa levá-lo TAMBÉM para a branch `release/v2.3`, sem trazer o resto do histórico.

git log main --oneline
# 7d8e9f0 fix: corrige vazamento de memória em Worker Thread <- só esse commit

git checkout release/v2.3
git cherry-pick 7d8e9f0
```

Retorno esperado:

```
[release/v2.3 3a4b5c6] fix: corrige vazamento de memória em Worker Thread
Date: ...
1 file changed, 8 insertions(+), 2 deletions(-)
```

Quando usar: aplicar um commit específico (geralmente hotfix ou correção pontual) em outra branch, sem fazer merge da branch inteira — comum quando você mantém múltiplas versões/releases em paralelo.

✨ Cenário de Falha em Produção

Cherry-pick aplica o **diff** do commit original, mas não tem nenhuma garantia de que o código ao redor daquele diff é idêntico entre as branches. Se **release/v2.3** já divergiu significativamente de **main** (dependências diferentes, refatorações), o cherry-pick de um hotfix pode aplicar "limpo" (sem conflito de merge) mas ainda assim quebrar em runtime, porque o contexto ao redor mudou de forma que o Git não consegue detectar automaticamente — por exemplo, o hotfix assume uma função auxiliar que só existe na versão mais nova do arquivo em **main**, ausente em **release/v2.3**.

🧠 Dica de Produção

Depois de um cherry-pick de hotfix crítico, **nunca** confie apenas em "o Git não reportou conflito" como sinal de segurança — rode a suíte de testes completa na branch de destino antes de fazer deploy, e idealmente valide manualmente em staging. Mantenha um registro (mensagem de commit, ou um label no sistema de tracking) de quais hotfixes foram cherry-picked para quais releases — em incidentes futuros, "esse bug já foi corrigido?" precisa de resposta rápida, e sem esse rastro você perde tempo relendo históricos de branches divergentes sob pressão.

Resolução de conflitos — estratégia prática

```
git merge feature/nova-validacao
# Auto-merging src/validacao.ts
# CONFLICT (content): Merge conflict in src/validacao.ts
# Automatic merge failed; fix conflicts and then commit the result.
```

```
// dentro de src/validacao.ts, o Git marca o conflito assim:
<<<<<<< HEAD
export function validar(cpf: string): boolean {
  return cpf.length === 11;
}
=====
export function validar(cpf: string): boolean {
  return /\d{11}$/.test(cpf);
}
>>>>>>> feature/nova-validacao
```

Estratégia prática de resolução:

1. `git status` para ver todos os arquivos em conflito.
2. Para cada arquivo, entenda a **intenção** de cada lado (não apenas escolha um "às cegas") — geralmente a versão certa combina os dois.
3. Remova as marcações (`<<<<<<<` , `=====` , `>>>>>>>`) deixando o código final correto.
4. `git add <arquivo>` para cada arquivo resolvido.
5. `git commit` (sem `-m` , o Git já sugere uma mensagem de merge) ou `git rebase --continue` se estava em um rebase.

Retorno esperado após resolver e commitar:

```
[main 8f7e6d5] Merge branch 'feature/nova-validacao'
```

 **DICA DE PLENO:** em caso de dúvida sobre qual lado do conflito está correto, use `git log --oneline <arquivo>` em cada branch para entender o contexto/intenção de cada mudança antes de decidir — nunca resolva conflitos "no chute".

🌟 Cenário de Falha em Produção

O pior tipo de resolução de conflito é a "silenciosa" — quando o Git consegue fazer **auto-merge** (sem marcar conflito explícito) mas o resultado semântico está errado. Exemplo real: duas branches alteram a mesma função de cálculo de frete em linhas diferentes (uma ajusta a fórmula, outra adiciona um novo parâmetro) — o Git faz merge automático sem pedir intervenção humana, porque as linhas não colidem literalmente, mas o resultado combinado é logicamente inconsistente (o novo parâmetro não é considerado na fórmula ajustada). Ninguém revisa isso com atenção porque "não teve conflito", e o bug só aparece em produção como cálculos de frete errados.

🧠 Dica de Produção

Um merge sem conflitos **não é garantia de correção semântica** — só de ausência de sobreposição textual. Para mudanças em áreas críticas (cálculo de preço, autenticação, permissões), o processo de revisão de PR deveria sempre revisar o **diff final da branch alvo** (não só o diff da feature isolada) antes de aprovar um merge que envolveu resolução de conflitos, e idealmente rodar a suíte de testes completa pós-merge, não confiar apenas no CI que rodou antes do merge acontecer.

7.3 Linux para Dev Back-end

```
# Ver se o processo Node está rodando e pegar o PID
ps aux | grep node
# usuario  12345  2.1  1.5  912345 123456 pts/0  Sl  10:32  0:15 node dist/main.js

# Descobrir qual processo está ocupando a porta 3000 (útil para "porta já em uso")
lsof -i :3000
# COMMAND  PID    USER  FD   TYPE DEVICE SIZE/OFF NODE NAME
# node     12345  usuario  20u  IPv4 123456      0t0  TCP *:3000 (LISTEN)

# Matar o processo pela porta, direto
kill -9 $(lsof -t -i:3000)

# Acompanhar logs em tempo real (aplicação rodando como serviço systemd)
journalctl -u minha-api -f
# ou, se os logs vão para um arquivo:
tail -f /var/log/minha-api/app.log

# Ajustar permissões de um script de deploy (torná-lo executável)
chmod +x deploy.sh

# Trocar o dono de um diretório de logs (comum após rodar como root acidentalmente)
chown -R node:node /var/log/minha-api

# Diagnóstico de uso de CPU/memória em tempo real (interativo)
htop
```

Retorno esperado de `lsof -i :3000` quando a porta está livre:

(sem saída — comando retorna vazio)



Por baixo dos panos

`kill -9` envia `SIGKILL`, que o processo **não pode capturar nem ignorar** — o kernel derruba o processo imediatamente, sem dar chance de rodar código de cleanup (fechar conexões de banco, terminar de escrever um arquivo, drenar requisições em andamento). Isso difere de `SIGTERM` (`kill -15`, o padrão sem flag), que o Node pode capturar via `process.on('SIGTERM', ...)` para fazer um **graceful shutdown**: parar de aceitar novas conexões, esperar as requisições em andamento terminarem, fechar pools de conexão, e só então sair.

🌟 Cenário de Falha em Produção

Um script de deploy (como o `deploy.sh` do exemplo) que usa `kill -9` para "matar rápido" o processo antigo antes de subir o novo interrompe requisições em andamento **no meio da execução** — um usuário que estava no meio de um checkout perde a conexão sem resposta alguma (nem erro, nem sucesso — a conexão simplesmente morre), e se a operação já tinha debitado o estoque mas não confirmado o pagamento, você pode ter inconsistência de dados que precisa de reconciliação manual depois.

🧠 Dica de Produção

Prefira sempre `kill -15` (SIGTERM, o padrão) e implemente um handler de graceful shutdown na aplicação: pare o servidor HTTP de aceitar novas conexões (`server.close()`), espere requisições em voo terminarem (com um timeout máximo, ex: 30s), feche pools de banco/Redis, e só então saia. Reserve `SIGKILL` como último recurso, com um timeout — a maioria dos orquestradores (Kubernetes, systemd) já segue esse padrão nativamente: enviam `SIGTERM` primeiro, esperam um `terminationGracePeriodSeconds` , e só então enviam `SIGKILL` se o processo não saiu sozinho. Se sua aplicação não trata `SIGTERM` , você está desperdiçando essa janela de graça e sofrendo o equivalente a um `kill -9` a cada deploy.

Script bash de deploy simples

```
#!/usr/bin/env bash
# deploy.sh
# chmod +x deploy.sh && ./deploy.sh

set -euo pipefail # para o script IMEDIATAMENTE em caso de erro, variável não definida, ou falha

echo "🔄 Buscando últimas mudanças..."
git pull origin main

echo "📦 Instalando dependências..."
npm ci --omit=dev

echo "🔧 Compilando TypeScript..."
npm run build

echo "🔍 Verificando se a porta 3000 está livre antes de reiniciar..."
if lsof -i :3000 > /dev/null; then
  echo "⚠️ Porta 3000 em uso — encerrando processo antigo..."
  kill -9 "$(lsof -t -i:3000)"
fi

echo "🚀 Iniciando aplicação..."
nohup node dist/main.js > app.log 2>&1 &

echo "✅ Deploy concluído. PID: $!"
```

Retorno esperado:

```
🔄 Buscando últimas mudanças...
📦 Instalando dependências...
🔧 Compilando TypeScript...
🔍 Verificando se a porta 3000 está livre antes de reiniciar...
🚀 Iniciando aplicação...
✅ Deploy concluído. PID: 54321
```

⚠️ **ARMADILHA COMUM:** esquecer `set -euo pipefail` no topo do script faz com que, se `npm run build` falhar por um erro de TypeScript, o script **continue executando** os passos seguintes mesmo assim — resultando em deploy de uma versão quebrada ou desatualizada.

🌟 Cenário de Falha em Produção

O script de deploy do exemplo tem um problema mais sutil que `set -euo pipefail` não resolve sozinho: ele mata o processo antigo **antes** de garantir que o novo subiu com sucesso (`nohup node dist/main.js & ... echo "Deploy concluído"` não verifica se o processo realmente ficou de pé, só que o comando de iniciar retornou). Se o `dist/main.js` novo tiver um erro de runtime que só aparece no boot (variável de ambiente faltando, por exemplo), o script "termina com sucesso" (`set -e` não pega isso, porque o `node` foi colocado em background com `&`), mas a API está, na verdade, fora do ar — um **deploy que se autodeclara bem-sucedido enquanto derruba produção**.

🧠 Dica de Produção

Todo script de deploy artesanal (não usando um orquestrador com rollback automático) precisa de uma etapa explícita de **verificação pós-deploy**: aguardar alguns segundos e fazer um `curl` no endpoint `/health` (seção 9.4) antes de declarar sucesso — e, criticamente, ter um plano de **rollback automático** se essa verificação falhar (manter o binário/imagem anterior disponível e trocar de volta). Scripts bash artesanais de deploy são o tipo de dívida técnica que funciona bem até o dia em que não funciona, geralmente às 3h da manhã; a essa altura da maturidade de uma startup com 50k usuários, migrar para um orquestrador com health check + rollback nativo (Kubernetes, ECS, ou mesmo PM2 com `pm2 deploy`) costuma valer o investimento.

🤔 **Desafio de Mentoria:** você recebe um alerta às 3h da manhã: a API está retornando 503 em `/health` (banco: ok, redis: falha). O deploy mais recente foi há 6 horas e não mexeu em nada relacionado a Redis. Usando só os comandos desta seção (`ps` , `lsof` , `journalctl / tail` , `htop`), descreva a sequência de passos que você seguiria nos primeiros 5 minutos para diagnosticar se é (a) o Redis caiu, (b) a rede entre a API e o Redis está com problema, ou (c) a própria API está com um vazamento de conexões para o Redis. O que cada comando te diria para diferenciar essas três hipóteses?

🔗 Referências

- [Docker Docs — Multi-stage builds](#)
- [Docker Compose — Healthchecks](#)
- [Git Docs — git rebase](#)
- [Git Docs — git cherry-pick](#)

- [Atlassian — Merging vs Rebasing](#)
-

8. Arquitetura e SOLID

8.1 SOLID com exemplos em Node.js/TypeScript

S — Single Responsibility Principle (Princípio da Responsabilidade Única)

❌ Jeito Júnior vs ✅ Jeito Pleno

```
// ❌ JEITO JÚNIOR — uma classe fazendo validação, persistência E envio de email
class UsuarioServiceJunior {
  async registrar(nome: string, email: string) {
    if (!email.includes('@')) throw new Error('Email inválido'); // validação
    // persistência direta com SQL cru
    console.log(`INSERT INTO usuarios (nome, email) VALUES ('${nome}', '${email}')`);
    // envio de email
    console.log(`Enviando email de boas-vindas para ${email}`);
  }
}
```

```
// ✅ JEITO PLENO — cada responsabilidade isolada em sua própria classe
class ValidadorEmail {
  validar(email: string): void {
    if (!email.includes('@')) throw new Error('Email inválido');
  }
}

class UsuarioRepository {
  async salvar(nome: string, email: string): Promise<void> {
    console.log(`[DB] Salvando usuário: ${nome}, ${email}`);
  }
}

class EmailService {
  async enviarBoasVindas(email: string): Promise<void> {
    console.log(`[EMAIL] Enviando boas-vindas para ${email}`);
  }
}

class UsuarioServicePleno {
  constructor(
    private validador: ValidadorEmail,
    private repo: UsuarioRepository,
    private emailService: EmailService
  ) {}

  async registrar(nome: string, email: string): Promise<void> {
    this.validador.validar(email);
    await this.repo.salvar(nome, email);
    await this.emailService.enviarBoasVindas(email);
  }
}
```

Por que a diferença: cada classe tem um **único motivo para mudar**. Se a regra de validação de email mudar, você mexe só em `ValidadorEmail` — sem risco de quebrar a lógica de persistência ou envio de email. Isso também torna cada peça testável isoladamente (mock de `EmailService` sem precisar mockar SQL).

🌟 Cenário de Falha em Produção — o outro lado do "Advogado do Diabo"

SRP aplicado sem critério também tem custo real: se `UsuarioServicePleno.registrar` precisa que **persistência e envio de email sejam transacionalmente consistentes** (ou ambos acontecem, ou nenhum), separar em três classes independentes **esconde** essa necessidade de coordenação. Um bug comum: o `repo.salvar` funciona, mas o `emailService.enviarBoasVindas` falha (serviço de email fora do ar) — o usuário fica cadastrado no banco, mas nunca recebe o email de confirmação, e não há nenhum mecanismo automático de retry ou compensação, porque cada classe "só faz sua parte" e ninguém orquestra a falha parcial. Separar responsabilidades não elimina a necessidade de pensar em consistência — só move essa responsabilidade para quem orquestra (`UsuarioServicePleno.registrar`), e é fácil esquecer disso.

🧠 Dica de Produção

Ao decompor uma operação em múltiplas classes de responsabilidade única, pergunte sempre: "o que acontece se o passo 2 falhar depois do passo 1 ter sucesso?". Em produção, isso aparece como dados órfãos ou inconsistentes — um `usuarioId` existente no banco sem o email de boas-vindas correspondente enviado, rastreável comparando logs de `[DB] Salvando usuário` versus `[EMAIL] Enviando boas-vindas` pelo mesmo identificador. Para operações que precisam de garantia "tudo ou nada" através de serviços diferentes, considere padrões como Saga (com compensação explícita) ou pelo menos uma fila com retry (enviar email de forma assíncrona e reprocessável, não síncrona dentro do fluxo de cadastro).

O — Open/Closed Principle (Aberto para extensão, fechado para modificação)

```
// npx ts-node ocp-example.ts

interface CalculadoraDeDesconto {
  calcular(valorOriginal: number): number;
}

class DescontoNatal implements CalculadoraDeDesconto {
  calcular(valorOriginal: number): number {
    return valorOriginal * 0.9; // 10% off
  }
}

class DescontoBlackFriday implements CalculadoraDeDesconto {
  calcular(valorOriginal: number): number {
    return valorOriginal * 0.5; // 50% off
  }
}

// Essa função NUNCA precisa ser modificada para suportar um novo tipo de desconto —
// basta criar uma nova classe que implemente CalculadoraDeDesconto.
function aplicarDesconto(valor: number, estrategia: CalculadoraDeDesconto): number {
  return estrategia.calcular(valor);
}

console.log(aplicarDesconto(100, new DescontoNatal()));
console.log(aplicarDesconto(100, new DescontoBlackFriday()));
```

Retorno esperado:

```
90
50
```

Por que importa: sem essa abstração, `aplicarDesconto` teria um `if/else` ou `switch` gigante que cresce (e precisa ser **modificado**) toda vez que surge uma nova promoção — violando o "fechado para modificação". Com a interface, o sistema fica **aberto para extensão** (nova classe) sem tocar em código existente e testado.

🌟 Cenário de Falha em Produção — quando OCP vira over-engineering

Aplicar OCP cedo demais, para um caso que "pode vir a ter" variações mas hoje só tem uma, gera indireção sem benefício — uma interface `CalculadoraDeDesconto` com uma única implementação real por 2 anos é complexidade paga antecipadamente por um futuro que talvez nunca chegue. O "advogado do diabo" aqui: se sua startup está no estágio de descobrir product-market fit, e a regra de desconto muda de forma imprevisível toda semana (não é "adicionar uma nova estratégia", é "mudar a estratégia existente"), a abstração de interface **atrapalha** — você acaba modificando a interface e todas as implementações mesmo assim, só que agora espalhado em mais arquivos.

🧠 Dica de Produção

Regra prática para decidir "abstrair agora ou depois": espere até ter **pelo menos duas variações reais** do mesmo conceito antes de introduzir a interface (a heurística "rule of three" adaptada). Em revisão de código, se você vê uma interface com uma única classe implementando ela e nenhum plano concreto de uma segunda implementação, questione se aquilo é preparação genuína ou "abstração por precaução" — a primeira economiza retrabalho futuro real, a segunda só adiciona arquivos para navegar durante um incidente às 3h da manhã, quando cada segundo de "onde está a lógica de verdade" importa.

D — Dependency Inversion Principle (Inversão de Dependência)

```
// npx ts-node dip-example.ts

// ❌ SEM DIP: o serviço depende diretamente de uma implementação concreta (Nodemailer)
// class NotificacaoServiceJunior {
//   private mailer = new Nodemailer(); // acoplamento direto – impossível trocar ou mockar facilmente
//   notificar(destinatario: string) { this.mailer.send(destinatario); }
// }

// ✅ COM DIP: o serviço depende de uma ABSTRAÇÃO, não de uma implementação concreta
interface CanalDeNotificacao {
  enviar(destinatario: string, mensagem: string): Promise<void>;
}

class NotificacaoPorEmail implements CanalDeNotificacao {
  async enviar(destinatario: string, mensagem: string): Promise<void> {
    console.log(`[EMAIL] Para ${destinatario}: ${mensagem}`);
  }
}

class NotificacaoPorSMS implements CanalDeNotificacao {
  async enviar(destinatario: string, mensagem: string): Promise<void> {
    console.log(`[SMS] Para ${destinatario}: ${mensagem}`);
  }
}

class NotificacaoService {
  // depende da INTERFACE, injetada via construtor – não sabe (nem precisa saber) qual implementação
  constructor(private canal: CanalDeNotificacao) {}

  async notificarPedidoEnviado(destinatario: string) {
    await this.canal.enviar(destinatario, 'Seu pedido foi enviado!');
  }
}

async function main() {
  const servicoEmail = new NotificacaoService(new NotificacaoPorEmail());
  const servicoSMS = new NotificacaoService(new NotificacaoPorSMS());

  await servicoEmail.notificarPedidoEnviado('ana@email.com');
  await servicoSMS.notificarPedidoEnviado('+55 41 99999-0000');
}

main();
```

Retorno esperado:

```
[EMAIL] Para ana@email.com: Seu pedido foi enviado!
[SMS] Para +55 41 99999-0000: Seu pedido foi enviado!
```

💡 **DICA DE PLENO:** "inversão de dependência" não significa "usar injeção de dependência" (isso é a técnica). O princípio é: **módulos de alto nível não devem depender de módulos de baixo nível — ambos devem depender de abstrações**. `NotificacaoService` (alto nível, regra de negócio) não conhece `Nodemailer` ou `Twilio` (baixo nível, detalhes de infra) — ele só conhece o contrato `CanalDeNotificacao`.

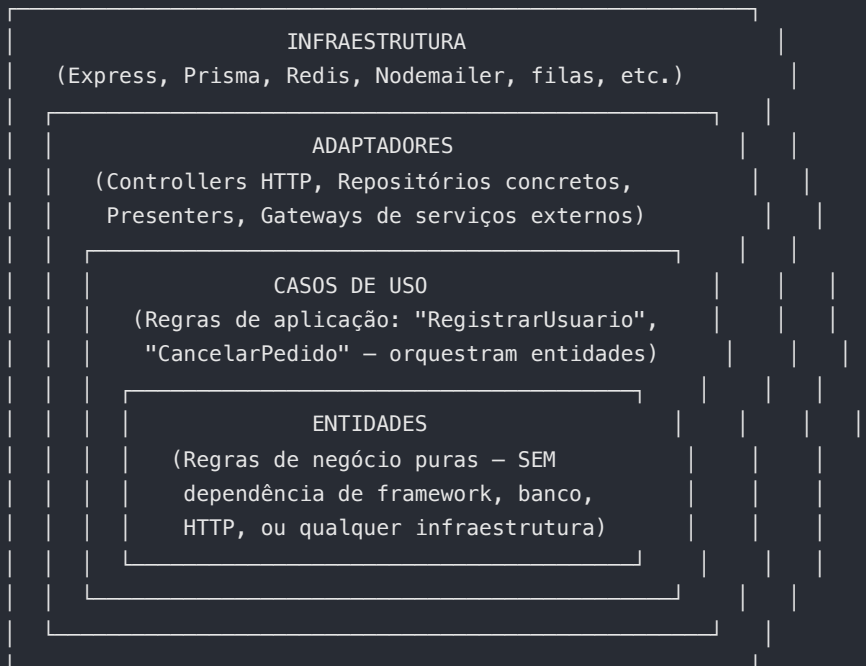
🔥 Cenário de Falha em Produção

DIP protege contra acoplamento, mas não protege contra **contratos mal desenhados**. Se `CanalDeNotificacao.enviar` retorna só `Promise<void>`, você perde informação crítica: o SMS falhou por número inválido (erro permanente, não adianta retentar) ou por indisponibilidade temporária do provedor (vale a pena retentar)? Com um contrato genérico demais, `NotificacaoService` não tem como decidir se deve colocar a notificação numa fila de retry ou descartar — a abstração "vazou" a decisão de negócio (retry ou não) para dentro dos detalhes de implementação de cada canal, que é exatamente o que DIP deveria evitar.

🧠 Dica de Produção

Ao desenhar uma interface/porta, pense em **quais decisões de negócio o chamador precisa tomar com o resultado** — se `NotificacaoService` precisa decidir sobre retry, o contrato precisa expor isso (ex: um tipo de erro `ErroTemporario` vs `ErroPermanente`, ou um enum de motivo de falha), não apenas "deu certo ou não". Em produção, monitore taxa de falha **por canal** (`notificacao_falhas_total{canal="sms"}` vs `{canal="email"}`) — se um canal específico tem taxa de falha crescente, isso geralmente indica problema no provedor externo (Twilio, SendGrid) e vale abrir um circuit breaker específico para aquele canal, sem afetar os demais.

8.2 Clean Architecture — conceitos e camadas



Regra de dependência: as setas SEMPRE apontam para dentro.
A camada de fora conhece a de dentro; o inverso NUNCA acontece.

```
// npx ts-node clean-architecture.ts

// === CAMADA: ENTIDADE (regra de negócio pura, zero dependência externa) ===
class Pedido {
  private itens: { nome: string; preco: number }[] = [];

  adicionarItem(nome: string, preco: number): void {
    if (preco <= 0) throw new Error('Preço deve ser positivo'); // regra de negócio pura
    this.itens.push({ nome, preco });
  }

  calcularTotal(): number {
    return this.itens.reduce((soma, item) => soma + item.preco, 0);
  }
}

// === CAMADA: CASO DE USO (orquestra entidades + portas/interfaces) ===
interface PedidoRepositoryPort {
  salvar(pedido: Pedido): Promise<void>;
}

class CriarPedidoUseCase {
  constructor(private repo: PedidoRepositoryPort) {}

  async executar(itens: { nome: string; preco: number }[]): Promise<number> {
    const pedido = new Pedido();
    itens.forEach((item) => pedido.adicionarItem(item.nome, item.preco));
    await this.repo.salvar(pedido);
    return pedido.calcularTotal();
  }
}

// === CAMADA: ADAPTADOR (implementação concreta da porta, sabe de infra) ===
class PedidoRepositoryEmMemoria implements PedidoRepositoryPort {
  async salvar(pedido: Pedido): Promise<void> {
    console.log('[INFRA] Pedido salvo. Total:', pedido.calcularTotal());
  }
}

// === CAMADA: INFRAESTRUTURA (o "detalhe" – Express, neste exemplo simulado) ===
async function controllerHttpSimulado() {
  const useCase = new CriarPedidoUseCase(new PedidoRepositoryEmMemoria());
  const total = await useCase.executar([
    { nome: 'Teclado', preco: 250 },
    { nome: 'Mouse', preco: 80 },
  ]);
  console.log('[HTTP] Resposta 201 – total do pedido:', total);
}

controllerHttpSimulado();
```

Retorno esperado:

```
[INFRA] Pedido salvo. Total: 330
[HTTP] Resposta 201 — total do pedido: 330
```

 **DICA DE PLENO:** a entidade **Pedido** **não importa nada** de Express, Prisma, ou HTTP — ela poderia ser reutilizada numa CLI, num worker de fila, ou num teste unitário puro, sem qualquer adaptação. É esse isolamento que faz a arquitetura ser "limpa": trocar o banco de dados ou o framework HTTP não deveria exigir tocar em uma linha sequer da entidade ou do caso de uso.

Cenário de Falha em Produção — o preço real do Clean Architecture

O "advogado do diabo" mais honesto aqui: Clean Architecture tem um custo de **indireção** que se paga em velocidade de desenvolvimento — uma mudança simples ("adicionar um campo **cupomDesconto** ao pedido") toca a entidade, o caso de uso, a porta (interface), o adaptador, e o controller HTTP. Em uma startup em estágio inicial, iterando rapidamente para achar product-market fit, esse custo de "tocar 5 arquivos para uma mudança simples" pode ser maior que o benefício de isolamento — Clean Architecture paga dividendos quando a lógica de negócio é complexa e estável, e a infraestrutura muda com frequência (trocar de Express para Fastify, de Postgres para outro banco); ela custa caro quando é o **negócio** que muda toda semana, não a infra.

Dica de Produção

Não existe "sempre use Clean Architecture" nem "nunca use" — a decisão depende de onde está a volatilidade do seu sistema. Uma heurística prática: se em 6 meses de operação sua equipe nunca trocou de banco de dados nem de framework HTTP, mas mudou regras de negócio centenas de vezes, a camada de "porta" para o banco pode estar sendo indireção sem retorno — considere simplificar. Do lado operacional, um sistema com Clean Architecture bem aplicada tem uma vantagem concreta em incidentes: como a entidade não depende de infraestrutura, você consegue reproduzir e testar a regra de negócio suspeita de um bug **sem precisar subir banco, Redis, ou qualquer serviço externo** — só instanciando a classe diretamente, o que acelera muito o ciclo de debug em produção.

8.3 Injeção de Dependência sem framework

```
// npx ts-node dependency-injection.ts

interface Logger {
  log(mensagem: string): void;
}

class ConsoleLogger implements Logger {
  log(mensagem: string): void {
    console.log(`[LOG] ${mensagem}`);
  }
}

class PedidoService {
  // Injeção via CONSTRUTOR — a forma mais comum e explícita
  constructor(private logger: Logger) {}

  processar(id: number) {
    this.logger.log(`Processando pedido ${id}`);
  }
}

// Injeção via FACTORY — útil quando a criação do objeto tem lógica condicional
function criarPedidoService(ambiente: 'producao' | 'teste'): PedidoService {
  const logger: Logger =
    ambiente === 'producao'
      ? new ConsoleLogger()
      : { log: () => {} }; // logger "mudo" em testes, evita poluir output

  return new PedidoService(logger);
}

const servico = criarPedidoService('producao');
servico.processar(101);
```

Retorno esperado:

```
[LOG] Processando pedido 101
```

 **DICA DE PLENO:** você não precisa de um framework de DI (como InversifyJS ou o container do NestJS) para praticar injeção de dependência — o princípio é simplesmente "não instancie suas dependências dentro da classe, receba-as de fora". Frameworks de DI só automatizam a **fiação** (wiring) dessas dependências em projetos grandes; o princípio arquitetural é o mesmo com ou sem eles.

🌟 Cenário de Falha em Produção

Um erro comum de composição manual (sem framework de DI): a `criarPedidoService` (função factory) esconde uma decisão de ambiente (`'producao'` vs `'teste'`) dentro do código de produção, em vez de vir de configuração externa. Se alguém adiciona um terceiro ambiente (`'staging'`) e esquece de atualizar essa factory, o `switch` /ternário cai no caso `else` implícito, potencialmente usando o logger "mudo" de teste **em produção** — silenciando logs importantes de um ambiente real sem que ninguém perceba, até faltar informação crítica durante um incidente.

🧠 Dica de Produção

Composição manual de dependências (sem framework) exige disciplina para não deixar decisões de ambiente "hardcoded" dentro do código de fiação — prefira ler de variáveis de ambiente validadas (`NODE_ENV` , com um valor default explícito e um erro se vier algo inesperado) em vez de um `switch` que assume só os casos conhecidos hoje. Em produção, se logs "somem" de um serviço específico sem erro nenhum, uma das primeiras hipóteses a checar é justamente essa: alguma lógica de composição de dependências resolvendo silenciosamente para uma implementação "muda" ou "de teste" por um `NODE_ENV` inesperado ou mal configurado no deploy.

🤔 **Desafio de Mentoria:** você está numa reunião de arquitetura e um colega propõe adotar Clean Architecture completa (as 4 camadas do diagrama da seção 8.2) para um novo microsserviço que vai apenas fazer proxy/agregação de 3 outras APIs internas, sem lógica de negócio própria significativa. Como "Advogado do Diabo", que perguntas você faria para essa proposta — e em que ponto a resposta muda se esse "simples proxy" tende historicamente, em startups de alto crescimento, a acumular lógica de negócio real com o tempo?

🔗 Referências

- [Robert C. Martin — The Clean Architecture \(artigo original\)](#)
- [DigitalOcean — SOLID Principles Explained](#)
- [Martin Fowler — Inversion of Control Containers and the Dependency Injection pattern](#)
- [NestJS Docs — Custom Providers \(referência prática de DI em Node\)](#)

9. Observabilidade

9.1 Logs estruturados com **pino**

❌ Jeito Júnior vs ✅ Jeito Pleno

```
// ❌ JEITO JÚNIOR — console.log sem estrutura, difícil de buscar/filtrar em produção
console.log('Usuário 42 fez login às ' + new Date());
console.log('Erro ao processar pedido:', erro);
```

```
// ✅ JEITO PLENO — logs estruturados (JSON), prontos para ferramentas de agregação
// npm install pino
// node pino-exemplo.js

const pino = require('pino');
const logger = pino({ level: 'info' });

logger.info({ usuarioId: 42, acao: 'login' }, 'Usuário fez login');
logger.warn({ pedidoId: 101, tentativas: 3 }, 'Retentativa de processamento de pedido');
logger.error({ err: new Error('Falha ao conectar no banco'), pedidoId: 101 }, 'Erro ao processar p
```

Retorno esperado (cada linha é um JSON, uma por evento — formato "ndjson"):

```
{"level":30,"time":1690000000000,"pid":12345,"hostname":"api-1","usuarioId":42,"acao":"login","msg
{"level":40,"time":1690000000010,"pid":12345,"hostname":"api-1","pedidoId":101,"tentativas":3,"msg
{"level":50,"time":1690000000020,"pid":12345,"hostname":"api-1","err":{"type":"Error","message":"F
```

Por que a diferença: **console.log** gera texto livre, difícil de indexar/buscar em ferramentas como Datadog, Elastic ou CloudWatch. Logs estruturados em JSON permitem **filtrar por campo** (**usuarioId:42**), agregar métricas (contar quantos **level:50** — erros — ocorreram por hora), e correlacionar eventos do mesmo **pedidoId** entre serviços diferentes.

⚠️ **ARMADILHA COMUM:** logar dados sensíveis (senha, token, número de cartão) em texto estruturado é ainda pior que em **console.log**, porque esses logs geralmente são **persistidos e replicados** em ferramentas de terceiros. Sempre mascare ou omita campos sensíveis antes de logar (**pino** suporta **redact** para isso automaticamente).



Por baixo dos panos

pino é rápido (uma das razões de sua popularidade em Node) porque evita serialização síncrona custosa no caminho crítico: por padrão, ele escreve para **stdout** de forma assíncrona e usa uma técnica de serialização otimizada (evita **JSON.stringify** genérico do V8 quando possível, usando serializadores pré-compilados para os campos comuns). Isso importa em alta escala: **console.log** é significativamente mais lento porque formata para humanos (cores, inspeção recursiva de objetos) a cada chamada — sob 50k usuários gerando milhares de logs por segundo, a diferença de overhead de logging pode se tornar mensurável na latência da aplicação.

🔥 Cenário de Falha em Produção

Logar `{ usuarioId: 42, dadosCompleto: usuario }` "para debug", onde **usuario** é o objeto inteiro vindo do banco, é uma forma comum e não intencional de vaziar dados sensíveis — se **usuario** tiver um campo **senhaHash** ou **tokenResetSenha**, ele vai parar no agregador de logs (Datadog, Elastic), replicado, indexado, e potencialmente acessível por qualquer pessoa do time com acesso de leitura aos logs — um vetor de vazamento de dados frequentemente esquecido em auditorias de segurança, porque "é só um log de debug".

🧠 Dica de Produção

Configure **redact** do pino com uma lista **exaustiva** de paths sensíveis (**senha** , ***.senhaHash** , ***.token** , **req.headers.authorization**) desde o início do projeto — adicionar depois de um vazamento já ter acontecido é tarde demais, porque os logs antigos já foram persistidos e replicados. Em produção, adicione uma checagem automatizada (linter customizado ou teste) que falha o build se um **logger.info** / **logger.error** receber diretamente um objeto de entidade completo (**usuario** , **pedido**) sem passar por um serializador explícito que só extrai os campos permitidos — previna a categoria do erro, não apenas a instância.

9.2 Métricas básicas com **prom-client**

Três métricas fundamentais para observabilidade de uma API: **contagem de requisições**, **duração** e **taxa de erro** — o famoso "RED method" (Rate, Errors, Duration).

```
// npm install prom-client express
// node metrics-prometheus.js

const client = require('prom-client');
const express = require('express');

const registro = new client.Registry();
client.collectDefaultMetrics({ register: registro }); // métricas de CPU/memória do processo, de g

const contadorRequisicoes = new client.Counter({
  name: 'http_requests_total',
  help: 'Total de requisições HTTP recebidas',
  labelNames: ['metodo', 'rota', 'status'],
  registers: [registro],
});

const duracaoRequisicoes = new client.Histogram({
  name: 'http_request_duration_seconds',
  help: 'Duração das requisições HTTP em segundos',
  labelNames: ['metodo', 'rota'],
  buckets: [0.05, 0.1, 0.3, 0.5, 1, 2, 5],
  registers: [registro],
});

const app = express();

app.use((req, res, next) => {
  const fimTimer = duracaoRequisicoes.startTimer({ metodo: req.method, rota: req.path });
  res.on('finish', () => {
    contadorRequisicoes.inc({ metodo: req.method, rota: req.path, status: res.statusCode });
    fimTimer();
  });
  next();
});

app.get('/produtos', (req, res) => res.json([ { id: 1, nome: 'Teclado' } ]));

// Endpoint que o Prometheus (ou similar) faz "scrape" periodicamente
app.get('/metrics', async (req, res) => {
  res.set('Content-Type', registro.contentType);
  res.end(await registro.metrics());
});

app.listen(3000, () => console.log('Métricas disponíveis em /metrics'));
```

Retorno esperado ao acessar **GET /metrics** (trecho relevante):

```
# HELP http_requests_total Total de requisições HTTP recebidas
# TYPE http_requests_total counter
http_requests_total{metodo="GET",rota="/produtos",status="200"} 3

# HELP http_request_duration_seconds Duração das requisições HTTP em segundos
# TYPE http_request_duration_seconds histogram
http_request_duration_seconds_bucket{metodo="GET",rota="/produtos",le="0.05"} 3
http_request_duration_seconds_sum{metodo="GET",rota="/produtos"} 0.012
http_request_duration_seconds_count{metodo="GET",rota="/produtos"} 3
```

💡 **DICA DE PLENO:** um **Histogram** (não apenas uma média) é essencial porque médias **escondem outliers**. Uma API com maioria das requisições em 50ms e uma minoria travando em 5s tem uma média enganosa e "aceitável" — o histograma revela a distribuição real, permitindo calcular percentis (p95, p99) que refletem a experiência real dos usuários mais afetados.

🌟 Cenário de Falha em Produção

O label **rota: req.path** do exemplo é uma armadilha de **cardinalidade não intencional**: se sua rota é **/produtos/:id**, **req.path** gera um valor único por ID (**/produtos/1**, **/produtos/2**, ...), e cada valor único de label cria uma nova série temporal no Prometheus. Com 50k usuários acessando milhares de produtos diferentes, você gera **centenas de milhares de séries temporais** para o que deveria ser uma única métrica agregada — isso é conhecido como "explosão de cardinalidade", e pode derrubar o próprio Prometheus (consumo de memória descontrolado) ou tornar o **/metrics** tão grande que o scrape periódico começa a falhar por timeout.

🧠 Dica de Produção

Sempre use o **padrão da rota** (**req.route?.path**, que no Express dá **/produtos/:id**, não o valor resolvido) como label, nunca o path literal com IDs. Se seu APM/Prometheus está lento para carregar dashboards ou o processo do Prometheus está com uso de memória crescente sem explicação óbvia, a primeira coisa a checar é cardinalidade de labels — rode uma query tipo **count by (__name__)(count without(instance) {__name__=~".+"})** para achar as métricas com mais séries e investigar se algum label (IDs, emails, UUIDs) está vazando para dentro de um label de métrica.

9.3 Monitoramento de erros com Sentry

```
// npm install @sentry/node
// npx ts-node sentry-exemplo.ts

import * as Sentry from '@sentry/node';

Sentry.init({
  dsn: 'https://exemplo@0000000.ingest.sentry.io/000000', // DSN do seu projeto no Sentry
  environment: process.env.NODE_ENV ?? 'development',
  tracesSampleRate: 0.2, // captura 20% das transações para performance tracing
});

async function processarPedido(id: number) {
  if (id < 0) throw new Error(`ID de pedido inválido: ${id}`);
  return { id, status: 'processado' };
}

async function main() {
  try {
    await processarPedido(-5);
  } catch (erro) {
    // Captura manual, com contexto extra útil para debugar
    Sentry.captureException(erro, {
      tags: { modulo: 'pedidos' },
      extra: { pedidoId: -5 },
    });
    console.log('Erro capturado e enviado ao Sentry (contexto: modulo=pedidos)');
  }
}

// Captura AUTOMÁTICA de exceções não tratadas (crash silencioso vira alerta)
process.on('uncaughtException', (erro) => {
  Sentry.captureException(erro);
  console.error('Exceção não tratada capturada pelo Sentry:', erro.message);
  process.exit(1); // após uma uncaughtException, o estado do processo é instável – sempre reinici
});

main();
```

Retorno esperado:

```
Erro capturado e enviado ao Sentry (contexto: modulo=pedidos)
```

(no dashboard do Sentry, o erro aparece agrupado com stack trace, tag `modulo:pedidos` e o `extra.pedidoId`)

⚠️ **ARMADILHA COMUM:** capturar a exceção com `Sentry.captureException` mas **não** relançar/tratar adequadamente pode mascarar o problema para o usuário (a requisição "aparenta" ter sucesso quando na verdade falhou). Monitoramento de erro não substitui tratamento de erro — são complementares.

Por baixo dos panos

O `process.exit(1)` depois de `uncaughtException` não é uma formalidade — é essencial. Depois de uma exceção não tratada, o estado interno do processo é considerado **não confiável**: closures podem estar em estados intermediários inconsistentes, listeners podem ter sido parcialmente removidos, o event loop pode ter callbacks pendentes de operações que nunca vão completar corretamente. Continuar rodando "porque capturamos o erro" é um antipadrão perigoso — o processo pode continuar respondendo requisições, mas com corrupção de estado sutil e imprevisível. Deixar o orquestrador (Kubernetes/PM2) reiniciar o processo do zero é a única forma confiável de garantir estado limpo.

💥 Cenário de Falha em Produção

Sem `tracesSampleRate` calibrado corretamente, um serviço com alto tráfego (50k usuários ativos) pode gerar centenas de milhares de eventos de tracing por hora — mesmo em 20% de amostragem. Isso não é só custo financeiro (a maioria dos planos do Sentry cobra por evento); em picos de tráfego real, o overhead de instrumentação (serialização de contexto, chamadas de rede para o Sentry) pode competir por recursos com o tráfego real da aplicação, ironicamente piorando a performance justamente durante o pico que você mais precisa observar de perto.

Dica de Produção

Ajuste `tracesSampleRate` dinamicamente por rota/criticidade (o SDK do Sentry suporta `tracesSampler` como função) — amostre 100% de erros e transações lentas, mas reduza drasticamente a amostragem de rotas de alto volume e baixo risco (ex: health checks, endpoints de métricas). Se o próprio processo de observabilidade está degradando a performance que ele deveria só observar, isso é um sinal de configuração inadequada para a escala atual — revise `tracesSampleRate` toda vez que o tráfego crescer significativamente, não apenas na configuração inicial do projeto.

9.4 Health Check endpoint

```
// npm install express ioredis pg
// npx ts-node health-check.ts

import express from 'express';
import Redis from 'ioredis';
import { Pool } from 'pg';

const app = express();
const redis = new Redis();
const pool = new Pool();

app.get('/health', async (req, res) => {
  const status = {
    api: 'ok',
    banco: 'desconhecido',
    redis: 'desconhecido',
  };

  try {
    await pool.query('SELECT 1');
    status.banco = 'ok';
  } catch {
    status.banco = 'falha';
  }

  try {
    await redis.ping();
    status.redis = 'ok';
  } catch {
    status.redis = 'falha';
  }

  const tudoOk = Object.values(status).every((s) => s === 'ok');
  res.status(tudoOk ? 200 : 503).json(status);
});

app.listen(3000, () => console.log('Health check disponível em /health'));
```

Retorno esperado (cenário saudável, **GET /health**, status HTTP 200):

```
{ "api": "ok", "banco": "ok", "redis": "ok" }
```

Retorno esperado (Redis fora do ar, status HTTP 503):

```
{ "api": "ok", "banco": "ok", "redis": "falha" }
```

 **DICA DE PLENO:** esse endpoint é exatamente o que o **HEALTHCHECK** do Dockerfile (seção 7.1) e os *liveness/readiness probes* do Kubernetes consultam para decidir se devem reiniciar o container ou tirar a instância do balanceamento de carga — um health check que só checa **api: ok** sem checar as dependências reais dá **falsos positivos** perigosos.

Cenário de Falha em Produção — o "advogado do diabo" do próprio health check

O exemplo do material, embora correto como conceito, tem um risco real em produção se usado como **liveness probe** (não apenas readiness): se o Redis cair, **/health** retorna 503, e se esse endpoint for o *liveness probe* do Kubernetes, ele vai **reiniciar o container da API repetidamente**, mesmo que a API em si esteja perfeitamente saudável — o problema é do Redis, não da API, mas você está matando o componente errado. Pior: se todas as réplicas da API fazem isso simultaneamente (porque compartilham o mesmo Redis), você pode causar uma **cascata de reinícios** que impede a API de voltar a ficar saudável mesmo depois do Redis se recuperar, porque todas as réplicas ficam presas num ciclo de boot-check-fail-restart.

Dica de Produção

Separe semanticamente **liveness** ("o processo está travado/corrompido e precisa reiniciar?") de **readiness** ("o serviço está pronto para receber tráfego agora?").

Dependências externas (banco, Redis) devem influenciar apenas o **readiness** — se o Redis cair, a API deveria sair do balanceamento de carga (parar de receber tráfego novo) sem ser reiniciada, porque reiniciar não vai trazer o Redis de volta, só vai causar downtime desnecessário e possivelmente um loop de crash. Reserve o *liveness probe* para checagens que só o próprio processo pode corrigir se reiniciado (ex: detectar que o event loop está travado há muito tempo, ou memória crescendo sem limite).

 **Desafio de Mentoria:** dos quatro pilares de observabilidade desta seção (logs, métricas, error tracking, health check), qual você implementaria **primeiro** numa API nova que ainda não tem nenhum? Justifique pensando no cenário mais provável de um incidente real numa startup em crescimento — e explique por que a ordem de prioridade muda (ou não) se a equipe é pequena (3 desenvolvedores) versus já tem um time de plataforma/SRE dedicado.

Referências

- [pino — Logger de alta performance](#)
 - [prom-client \(npm\) — Métricas para Prometheus](#)
 - [Prometheus Docs — Metric Types](#)
 - [Sentry Docs — Node.js](#)
 - [Kubernetes Docs — Liveness, Readiness and Startup Probes](#)
-

10. Performance e Profiling

10.1 `node --inspect` + Chrome DevTools: heap snapshot e memory leaks

```
// node --inspect memory-leak-exemplo.js

// Exemplo PROPOSITAL de memory leak: um array que cresce indefinidamente
// e nunca é limpo, referenciado por um closure de setInterval que nunca para.
const cacheSemLimite = [];

setInterval(() => {
  cacheSemLimite.push({ dados: new Array(10000).fill('vazando memória') });
  console.log('Itens no cache:', cacheSemLimite.length);
}, 100);
```

Passo a passo para investigar:

1. Rode com `node --inspect memory-leak-exemplo.js` — o terminal mostra uma URL tipo `chrome://inspect`.
2. Abra o Chrome, acesse `chrome://inspect`, clique em "inspect" sob o processo Node listado.
3. Na aba **Memory** do DevTools, clique em "Take heap snapshot".
4. Espere alguns segundos (deixe o "vazamento" rodar), tire um **segundo snapshot**.
5. Compare os dois snapshots usando o filtro "**Comparison**" — objetos que só crescem entre snapshots (nunca são coletados pelo Garbage Collector) são candidatos fortes a memory leak.

Retorno esperado no terminal (o processo roda, mas a memória do processo cresce sem parar):

```
Debugger listening on ws://127.0.0.1:9229/a1b2c3d4-...
For help, see: https://nodejs.org/en/docs/inspector
Itens no cache: 1
Itens no cache: 2
Itens no cache: 3
...
Itens no cache: 500 (RSS do processo cresce continuamente – visível no Activity Monitor / http)
```

⚠ **ARMADILHA COMUM:** o padrão clássico de memory leak em Node.js é um array, `Map`, ou `Set` **global** (ou em closure de longa duração, como um `setInterval` que nunca é limpo) que recebe itens continuamente sem nunca remover os antigos — muito comum em implementações "caseiras" de cache sem TTL/limite de tamanho (compare com a seção 4.3, que resolve exatamente isso usando Redis com TTL).

Por baixo dos panos

O V8 usa um Garbage Collector **generacional**: objetos novos vão para a "young generation" (coletada frequentemente e rapidamente via algoritmo *Scavenge*), e objetos que sobrevivem a múltiplos ciclos são promovidos para a "old generation" (coletada com menos frequência, via *Mark-Sweep-Compact*, mais custoso). Um memory leak real (referência viva que nunca é liberada) faz objetos serem promovidos para old generation e nunca coletados — o GC não é "burro", ele simplesmente **não pode** coletar algo que ainda tem uma referência alcançável a partir das raízes (variáveis globais, closures ativas, o call stack). O GC de old generation rodando com mais frequência conforme o heap cresce é, por si só, uma fonte de latência: pausas de "stop-the-world" (embora o V8 moderno minimize isso com coleta incremental/concorrente) aparecem como picos de latência p99 correlacionados com o crescimento de memória.

✨ Cenário de Falha em Produção

Diferente do exemplo didático (um array óbvio crescendo), memory leaks reais em produção costumam ser mais sutis: um `Map` usado como cache de sessões de WebSocket, onde a chave é removida ao desconectar — **exceto** quando a desconexão acontece por erro de rede abrupto (não pelo evento `'close'` limpo), caso em que o listener de cleanup nunca dispara e a entrada fica presa no `Map` para sempre. Sob 50k usuários com conexões instáveis (redes móveis, por exemplo), esse tipo de vazamento cresce lentamente ao longo de dias — o processo funciona bem por semanas e então, sem nenhum deploy recente, começa a reiniciar sozinho por OOM (Out Of Memory) em horários aparentemente aleatórios.

 Dica de Produção

Memory leaks lentos são identificáveis no APM/Grafana por um padrão de **dente de serra** no gráfico de memória: crescimento constante e linear ao longo de horas/dias, seguido de uma queda abrupta (o processo reiniciou, por OOM kill do orquestrador ou crash). Se o intervalo entre quedas está diminuindo (o processo demora cada vez menos para vazar até o limite), o leak provavelmente está correlacionado com volume de tráfego. O diagnóstico definitivo continua sendo heap snapshot comparativo em produção (com cuidado: tirar um heap snapshot pausa o processo por um tempo, então faça isso numa réplica isolada do tráfego real, nunca direto no serviço principal sob carga) — procure por contagem de instâncias de um mesmo shape/classe crescendo sem parar entre dois snapshots.

10.2 Teste de carga simples com **autocannon**

```
# npm install -g autocannon
# (assumindo uma API rodando em http://localhost:3000)

autocannon -c 50 -d 10 http://localhost:3000/produtos
# -c 50: 50 conexões simultâneas
# -d 10: por 10 segundos
```

Retorno esperado:

```
Running 10s test @ http://localhost:3000/produtos
50 connections
```

Stat	2.5%	50%	97.5%	99%	Avg	Stdev	Max
Latency	8 ms	12ms	45 ms	60ms	14.2 ms	9.87 ms	120ms

Stat	1%	2.5%	50%	97.5%	Avg	Stdev	Min
Req/Sec	3021	3021	3542	3789	3489.2	210.4	3021

```
34.9k requests in 10.03s, 5.2 MB read
```

💡 **DICA DE PLENO:** olhe sempre para o **p97.5/p99** (percentis altos), não só a média — são eles que representam a experiência dos usuários "de sorte azarada" (ex: uma requisição que coincide com uma pausa do Garbage Collector). Uma média de 14ms com p99 de 60ms ainda pode ser aceitável; um p99 de 2000ms é um sinal de alerta mesmo com média baixa.

🔥 Cenário de Falha em Produção

Rodar **autocannon** contra **localhost** na sua própria máquina de desenvolvimento dá números **enganosos** — sem latência de rede real, sem contenção de recursos compartilhados (banco de dados sob carga real de outros serviços), sem o comportamento real do connection pool sob concorrência. Um Pleno testa localmente, vê 3500 req/s, aprova a mudança como "performática o suficiente" — mas em produção, atrás de um load balancer, com o banco compartilhado por outros serviços e latência de rede real entre containers, o throughput real cai para uma fração disso, e ninguém percebeu porque o teste de carga nunca rodou contra um ambiente representativo.

🧠 Dica de Produção

Testes de carga só têm valor preditivo real quando rodados contra um ambiente que se aproxima de produção (staging com dados de volume realista, mesma topologia de rede, mesmo tamanho de connection pool) — nunca contra **localhost** isolado. Em produção de verdade, complemente testes de carga sintéticos com **testes de carga em produção controlados** (canary releases, ou ferramentas de "shadow traffic" que espelham tráfego real para uma nova versão sem afetar usuários) antes de um lançamento que espera pico de tráfego significativo (ex: uma campanha de marketing) — números de **autocannon** local são um sinal de fumaça, não uma garantia.

10.3 Dicas práticas de performance

JSON.parse / JSON.stringify desnecessários

```
// node performance-json.js

// ❌ JEITO JÚNIOR – clona um objeto via stringify+parse (lento, e quebra tipos como Date)
function clonarObjetoJunior(obj) {
  return JSON.parse(JSON.stringify(obj));
}

// ✅ JEITO PLENO – structuredClone (nativo desde Node 17+), mais rápido e preserva tipos
function clonarObjetoPleno(obj) {
  return structuredClone(obj);
}

const original = { nome: 'Ana', criadoEm: new Date(), tags: new Set(['vip']) };

const clone1 = clonarObjetoJunior(original);
const clone2 = clonarObjetoPleno(original);

console.log('clone via JSON – criadoEm é Date?', clone1.criadoEm instanceof Date);
console.log('clone via structuredClone – criadoEm é Date?', clone2.criadoEm instanceof Date);
console.log('clone via JSON – tags é Set?', clone1.tags instanceof Set);
console.log('clone via structuredClone – tags é Set?', clone2.tags instanceof Set);
```

Retorno esperado:

```
clone via JSON – criadoEm é Date? false
clone via structuredClone – criadoEm é Date? true
clone via JSON – tags é Set? false
clone via structuredClone – tags é Set? true
```

⚠️ **ARMADILHA COMUM:** `JSON.parse(JSON.stringify(obj))` é um "truque" comum para clonar objetos, mas **silenciosamente destrói** `Date` (vira string), `Set` / `Map` (viram `{}`), `undefined` (é removido), e funções. Prefira `structuredClone()` (nativo) para clonagem profunda segura.

🌟 Cenário de Falha em Produção

Um serviço que "clona" um objeto de configuração de pedido via

`JSON.parse(JSON.stringify(pedido))` para criar um snapshot antes de uma operação de retry perde silenciosamente o campo `criadoEm: Date` (vira `string ISO`). Se código posterior faz `pedido.criadoEm.getTime()` assumindo que ainda é um objeto `Date`, o processo lança `TypeError: pedido.criadoEm.getTime is not a function` — mas só no fluxo de retry, que roda raramente e pode não estar coberto por teste, então esse bug pode ficar dormiente em produção por meses até o primeiro retry real acontecer, geralmente durante um incidente que já está causando outros problemas, agravando o cenário.

🧠 Dica de Produção

`structuredClone` também tem uma limitação importante que vale saber antes de confiar cegamente: ele lança exceção ao tentar clonar funções ou determinados objetos não serializáveis (como conexões de banco, sockets) — se seu objeto de domínio acidentalmente carrega uma referência a algo assim (comum quando entidades de domínio são "contaminadas" com referências de infraestrutura, o que Clean Architecture, seção 8.2, existe para evitar), `structuredClone` falha de forma mais clara que o `JSON.stringify` (que silenciosamente omite funções) — na prática, isso é uma vantagem de diagnóstico: prefira falhar ruidosamente a falhar silenciosamente.

Set / Map em vez de Array.includes em loops

```
// node performance-set-vs-array.js

const idsPermitidosArray = Array.from({ length: 100000 }, (_, i) => i);
const idsPermitidosSet = new Set(idsPermitidosArray);

console.time('Array.includes (O(n) por busca)');
for (let i = 0; i < 10000; i++) {
  idsPermitidosArray.includes(99999); // busca linear – percorre o array inteiro toda vez
}
console.timeEnd('Array.includes (O(n) por busca)');

console.time('Set.has (O(1) por busca)');
for (let i = 0; i < 10000; i++) {
  idsPermitidosSet.has(99999); // busca por hash – praticamente instantânea
}
console.timeEnd('Set.has (O(1) por busca)');
```

Retorno esperado (valores ilustrativos — a proporção é o que importa):

```
Array.includes (O(n) por busca): 850.234ms
Set.has (O(1) por busca): 1.812ms
```

💡 **DICA DE PLENO:** `Array.includes` / `Array.find` fazem busca **linear** — o tempo cresce proporcionalmente ao tamanho do array. `Set.has` / `Map.get` usam hashing e são **O(1)** (tempo constante, independente do tamanho). Se você faz buscas repetidas de "esse ID está numa lista de permitidos?" dentro de um loop ou de uma rota chamada com frequência, essa troca é uma das otimizações de maior impacto com menor esforço de implementação.

💣 Cenário de Falha em Produção

Esse tipo de bug de complexidade algorítmica é traiçoeiro porque **não aparece em desenvolvimento nem em teste com poucos dados** — uma lista de 50 itens com `Array.includes` é instantânea, indistinguível de `Set.has` a olho nu. O problema só se manifesta quando o dado de produção cresce organicamente: uma lista de "IDs bloqueados" que começa com 10 itens e, dois anos depois, tem 80 mil, transformando uma rota que sempre foi "rápida o suficiente" numa rota que degrada **gradualmente**, sem nenhum deploy correspondente ao momento em que a lentidão começou a ficar perceptível — dificultando encontrar "o que mudou" porque, tecnicamente, nada mudou no código.

🧠 Dica de Produção

Esse padrão de degradação gradual sem deploy correspondente é reconhecível no APM como uma **tendência lenta de piora** na latência de uma rota específica ao longo de semanas/meses, não um degrau abrupto (que seria sinal de deploy ou de um evento externo). Se você notar isso, correlacione a latência da rota com o **tamanho da estrutura de dados envolvida** (uma métrica customizada tipo `tamanho_lista_permitidos` ajuda muito aqui) — se a latência cresce junto com o tamanho de uma lista/array em memória, você achou uma busca $O(n)$ que deveria ser $O(1)$, o exato padrão descrito nesta seção.

Streaming vs Buffer completo

Revisitando a seção 3.1: ao processar arquivos grandes (upload de CSV, exportação de relatório), **sempre** prefira streams a carregar o arquivo inteiro em memória com `fs.readFileSync`. Um arquivo de 2GB lido de uma vez pode facilmente estourar o limite de heap padrão do V8 (`--max-old-space-size`), derrubando o processo inteiro — enquanto a mesma operação via

stream processa em chunks de poucos KB por vez, com uso de memória constante independente do tamanho do arquivo.

🌟 Cenário de Falha em Produção

Um endpoint de "exportar todos os pedidos em CSV" implementado com `fs.readFileSync` /carregando tudo num array em memória funciona perfeitamente enquanto a base de clientes é pequena. Conforme a startup cresce para 50k usuários ativos (e, digamos, milhões de pedidos históricos), esse mesmo endpoint — nunca modificado, porque "sempre funcionou" — começa a derrubar o processo com **FATAL ERROR: Reached heap limit, Allocation failed – JavaScript heap out of memory** durante os horários de maior uso, exatamente quando o processo tem menos memória de sobra para absorver o pico. E porque é $O(n)$ com o crescimento da base de dados (não com tráfego), o incidente parece "aparecer do nada" um dia, sem relação óbvia com nenhum deploy.

🧠 Dica de Produção

Erros de **JavaScript heap out of memory** matam o processo Node imediatamente e sem chance de log gracioso — o V8 aborta o processo assim que detecta que não consegue alocar mais memória, então o log estruturado da sua aplicação frequentemente não captura nada além do stack trace nativo do V8 no `stderr` /log do container. Configure alertas de `nodejs.heap_size_used` (ou `process.memoryUsage().heapUsed` exposto como métrica) com threshold de crescimento anômalo correlacionado com uma rota específica, não apenas um limite absoluto — e, criticamente, audite periodicamente o código em busca de `readFileSync`, `.find()` sem paginação, ou `SELECT *` sem LIMIT em endpoints de exportação/relatório, porque esse é o padrão que mais recorrentemente causa esse tipo de incidente conforme uma startup escala.

🤔 **Desafio de Mentoria:** você tem um orçamento limitado de tempo de engenharia neste sprint e precisa escolher uma área desta seção 10 para investir primeiro: (a) configurar profiling contínuo de memória em produção, (b) rodar uma bateria de testes de carga em staging antes do próximo grande lançamento de marketing, ou (c) fazer uma auditoria de código procurando por padrões $O(n)$ escondidos (`Array.includes` em loops, `JSON.parse` / `stringify` para clone) nas rotas mais usadas. Dado que sua API tem 50k usuários ativos hoje e a empresa espera dobrar essa base nos próximos 6 meses, qual você escolhe primeiro, e que evidência (métrica, log, ou ausência dela) você buscaria agora para validar ou refutar essa escolha antes de comprometer o sprint inteiro nela?

Referências

- [Node.js Docs — Debugging Guide \(--inspect\)](#)
 - [Chrome DevTools — Memory panel](#)
 - [autocannon \(GitHub\)](#)
 - [clinic.js — Diagnóstico de performance para Node.js](#)
 - [MDN: structuredClone\(\)](#)
-

Conclusão

Esse guia cobre os pontos onde a diferença entre "sabe usar" e "sabe explicar o porquê" costuma aparecer em revisões de código, entrevistas técnicas e — principalmente — em produção, quando as coisas dão errado às 2h da manhã. Sugestão de uso:

1. Rode **todos** os exemplos de código você mesmo — não apenas leia.
2. Para cada seção "❌ Jeito Júnior vs ✅ Jeito Pleno", tente explicar em voz alta o "porquê" antes de ler a explicação.
3. Depois de estudar as seções 7 a 10 (Docker/Git/Linux, SOLID/Arquitetura, Observabilidade, Performance), volte às seções 1 a 3 — os fundamentos de runtime fazem mais sentido com o contexto operacional internalizado.
4. Use a tabela de referências abaixo como trilha de aprofundamento contínuo, não como leitura única.

 **DICA DE PLENO final:** a pergunta que separa Pleno de Sênior não é "você sabe a resposta?", é "você sabe em que situação essa resposta muda, e o que fazer quando o sistema real não se comporta como o exemplo do livro?". Estude sempre pensando em **trade-offs** e em **como você vai debugar isso em produção**, não em respostas absolutas.

Tabela Consolidada de Referências

Tópico	Links recomendados
Event Loop & Assincronismo	Node.js Event Loop, Timers, and process.nextTick() · Jake Archibald: In The Loop
Operadores modernos JS	MDN: Optional chaining (?.) · MDN: Nullish coalescing (??)
Closures	MDN: Closures
Promises	MDN: Promise.allSettled()
TypeScript Avançado	TypeScript Handbook — Utility Types · TypeScript Handbook — Generics · TypeScript Handbook — Narrowing · TS 4.9 — satisfies · Type Challenges
Streams & Buffers	Node.js Streams (docs) · Node.js Buffer (docs)
Worker Threads & Child Process	Node.js Worker Threads (docs) · Node.js Child Process (docs)
Padrões de Projeto	Node.js EventEmitter (docs) · Refactoring Guru — Design Patterns
Repository, Erros, Cache	Prisma Docs · Express — Error Handling · Redis — Caching Patterns
Testes	Vitest — Mocking · Supertest (GitHub) · fast-check · Martin Fowler — Unit Test
Segurança	OWASP — SQL Injection Prevention · bcrypt (npm) · jwt.io — Introdução ao JWT · node-postgres — Parameterized Query
Docker	Docker Docs — Multi-stage builds · Docker Compose — Healthchecks
Git	Git Docs — git rebase · Git Docs — git cherry-pick · Atlassian — Merging vs Rebasing
SOLID & Arquitetura	Uncle Bob — The Clean Architecture · DigitalOcean — SOLID Principles · Martin Fowler — Dependency Injection · NestJS — Custom Providers
Observabilidade	pino · prom-client · Prometheus — Metric Types · Sentry Docs — Node.js · Kubernetes — Probes
Performance & Profiling	Node.js — Debugging Guide · Chrome DevTools — Memory panel · autocannon · clinic.js · MDN: structuredClone()